

Zadanie 1: optymalizacja gradientowa

Adam Tokarz, Serhii Zeliuk

EAIIB ISI Metody i algorytmy optymalizacji grupa laboratoryjna 2 poniedziałek 18:30

Termin realizacji: 17 marca 2025

Wybierz jedną z testowych funkcji o dwóch zmiennych do optymalizacji (z https://en.wikipedia.org/wiki/Test_functions_for_optimization) (oprócz "sphere function", "Matyas function" i "Booth function").

Zadanie do oddania przez MS Teams. Do oddania: kod oraz krótkie sprawozdanie w PDF (można na przykład przy użyciu `quarto render notebook.ipynb --to pdf`).

Na 3.0

Do realizacji:

1. Weź implementację optymalizacji gradientowej z zadania z ostatnich ćwiczeń.
2. Wylosuj 10 punktów z dziedziny przeszukiwania z tabelki. Dla każdego z nich przeprowadź 100 kroków optymalizacji metodą największego spadku ze stałym krokiem. Narysuj wykres zależności wartości funkcji optymalizowanej od kroku optymalizacji.
3. Przeprowadź procedurę dla kilku kroków. Spróbuj zilustrować brak zbieżności, szybką zbieżność i powolną zbieżność.

Na 4.0

Do realizacji:

1. Punkty z zadania na 3.0.
2. Zamień stały krok z poprzedniego zadania na warunek wystarczającego spadku Armijo. Jako początkowy bracket przyjmij $[0, 1]$. Porównaj zbieżność metody o stałym kroku z przeszukiwaniem liniowym za pomocą wykresów wartości w konkretnym kroku uśrednione między poszczególnymi punktami początkowymi. Porównaj wyniki dla 6 różnych kombinacji parametrów metody Armijo.

Na 5.0

Do realizacji:

1. Punkty z zadania na 4.0.
2. Zamień stały bracket $[0, 1]$ z poprzedniego punktu na metodę automatycznie określającą bracket (przykład z wykładu). Porównaj wyniki z poprzednią metodą dla

minimum 6 zestawów parametrów bracketingu.

1.1 Przygotowanie funkcji

```
In [1]: using LinearAlgebra
        using Plots
        using Random
```

Jako jedną z testowych funkcji dwóch zmiennych do optymalizacji wybraliśmy Three-hump camel function, posiada 3 minima lokalne, w tym 1 globalne pośrodku.

`f(vec)` - Funkcja zwraca wynik funkcji Three-hump camel function na podstawie podanego wektora argumentów

`f_grad(vec)` - Funkcja oblicza gradient, pochodne cząstkowe funkcji w dwóch kierunkach, zwraca listę dwuelementową, pierwszy element to pochodna cząstkowa po x , drugi po y

```
In [2]: function f(vec)
        x, y = vec[1], vec[2]
        return 2 * x^2 - 1.05 * x^4 + x^6 / 6 + x*y + y^2
    end

    function f_grad(vec)
        x, y = vec[1], vec[2]
        return [4 * x - 4.2 * x^3 + x^5 + y, x + 2 * y]
    end
```

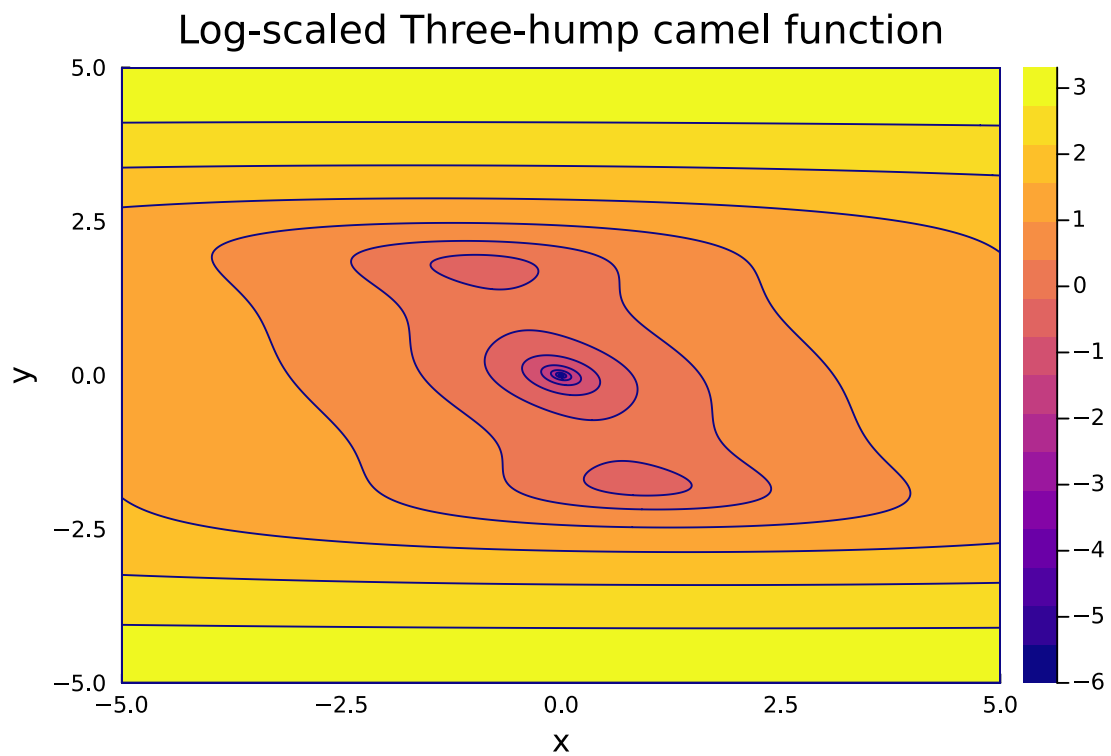
`f_grad` (generic function with 1 method)

1.2 Wizualizacja funkcji

Obliczanie wartości funkcji dla wartości x i y w przedziale $[-5, 5]$

```
In [3]: x = -5:0.01:5
        y = -5:0.01:5
        z = zeros(length(x), length(y))
        for i in 1:length(x)
            for j in 1:length(y)
                z[i, j] = f([x[i], y[j]])
            end
        end
```

```
In [4]: # Logarytmiczne skalowanie wartości funkcji w celu lepszej wizualizacji
        z_log = log10.(abs.(z) .+ 1e-6)
        contourf(x, y, z_log, fill=true, color=:plasma, xlabel="x", ylabel="y", title="L
```



1.3 Losowanie punktów

`rand_uniform(a, b)` - Funkcja losująca wartości z przedziału $[a, b]$

`points` - lista z 10 wylosowanymi punktami z przedziału $[-5, 5]$ dla x i y

```
In [5]: function rand_uniform(a, b)
        return rand() * (b-a) + a
    end

    # lista z 10 wylosowanymi punktami
    points = [[rand_uniform(-5, 5), rand_uniform(-5, 5)] for i in 1:10]
    points
```

```
10-element Vector{Vector{Float64}}:
 [0.23955296566897744, 3.1284380371411853]
 [3.1020637282223884, 3.7229446369452113]
 [4.6444624053469, 1.7980972343188775]
 [-3.9178383109187607, 2.7914166503996354]
 [4.018336027035277, -4.7087135619997404]
 [2.470290181516668, -3.792250265431385]
 [2.9199118270775486, -1.583917461936406]
 [-3.818182992350636, -4.291105867913095]
 [-1.5041804058230355, 4.149885299727195]
 [-4.864320645615063, -0.4338326129641912]
```

1.4 Metoda największego spadku

`steepest_gradient_descent(cost, grad, x0,  $\alpha$ ; max_iter=100, tol=1e-6)` - Funkcja implementująca algorytm metody największego spadku - szuka minimum globalnego funkcji. Funkcja na podstawie obliczonych gradientów funkcji f w każdej iteracji robi krok

w kierunku minimum. Funkcja kończy działanie gdy znajdzie się blisko minimum lub gdy zmiany będą niewielkie. Zwraca znaleziony wektor współrzędnych [w tym przykładzie współrzędne x i y] (minimum) oraz wektor wartości funkcji optymalizowanej dla kolejnych iteracji.

Argumenty:

- cost - funkcja kosztu, w tym przypadku jest to funkcja f , której wartość minimalizujemy
- grad - gradient funkcji kosztu, w tym przykładzie jest funkcja f_grad obliczająca gradienty funkcji f
- x_0 - punkt początkowy, od którego rozpoczyna się algorytm optymalizacji
- α - wartość kroku, przez który mnożony jest gradient
- max_iter - maksymalna liczba iteracji
- tol - współczynnik tolerancji, powoduje wyjście z pętli, gdy zmiany wartości funkcji kosztu są niewielkie

```
In [6]: function steepest_gradient_descent(cost, grad, x0,  $\alpha$ ; max_iter=100, tol=1e-6)
     $\theta$  = copy(x0)
    f_values = []
    storage = zeros(length( $\theta$ ))
    for i in 1:max_iter
        value_start = cost( $\theta$ )
        storage = grad( $\theta$ )
        norm_storage = norm(storage)
        if norm_storage == 0
            break
        end
         $\theta\_new$  =  $\theta$  - storage .* ( $\alpha$  / norm_storage)
        value_stop = cost( $\theta\_new$ )
        push!(f_values, value_start)
        if abs(value_stop - value_start) < tol
            break
        else
             $\theta$  =  $\theta\_new$ 
        end
    end
    return  $\theta$ , f_values
end
```

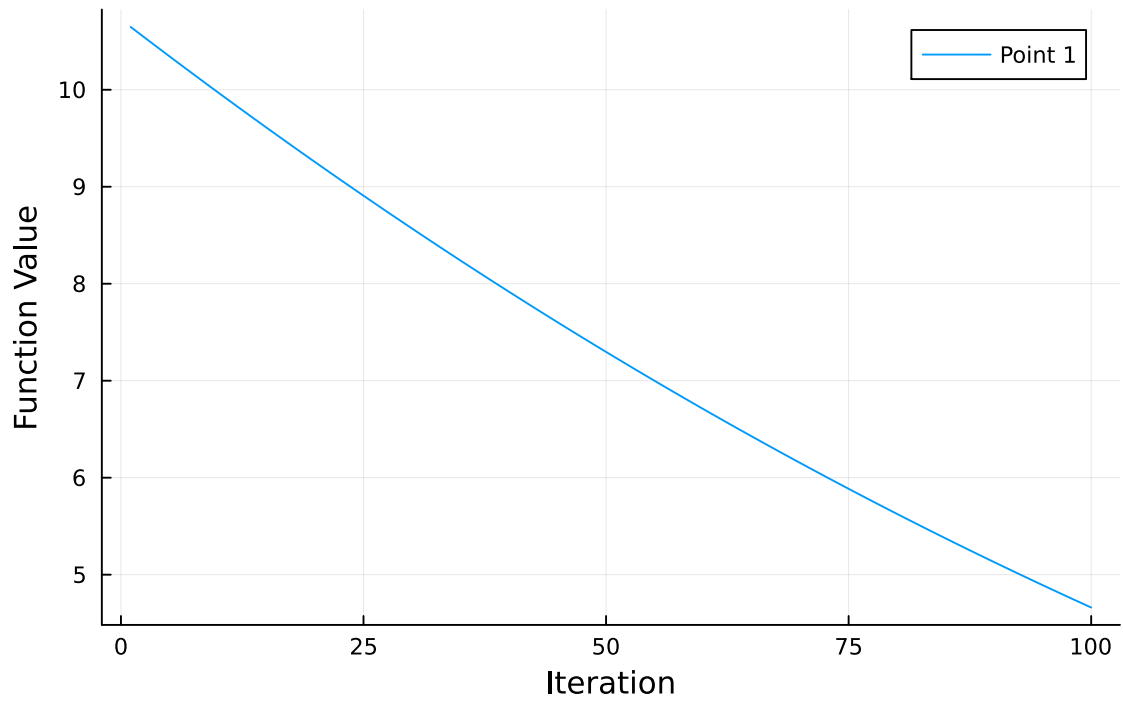
steepest_gradient_descent (generic function with 1 method)

1.5 Wyniki dla różnych kroków α

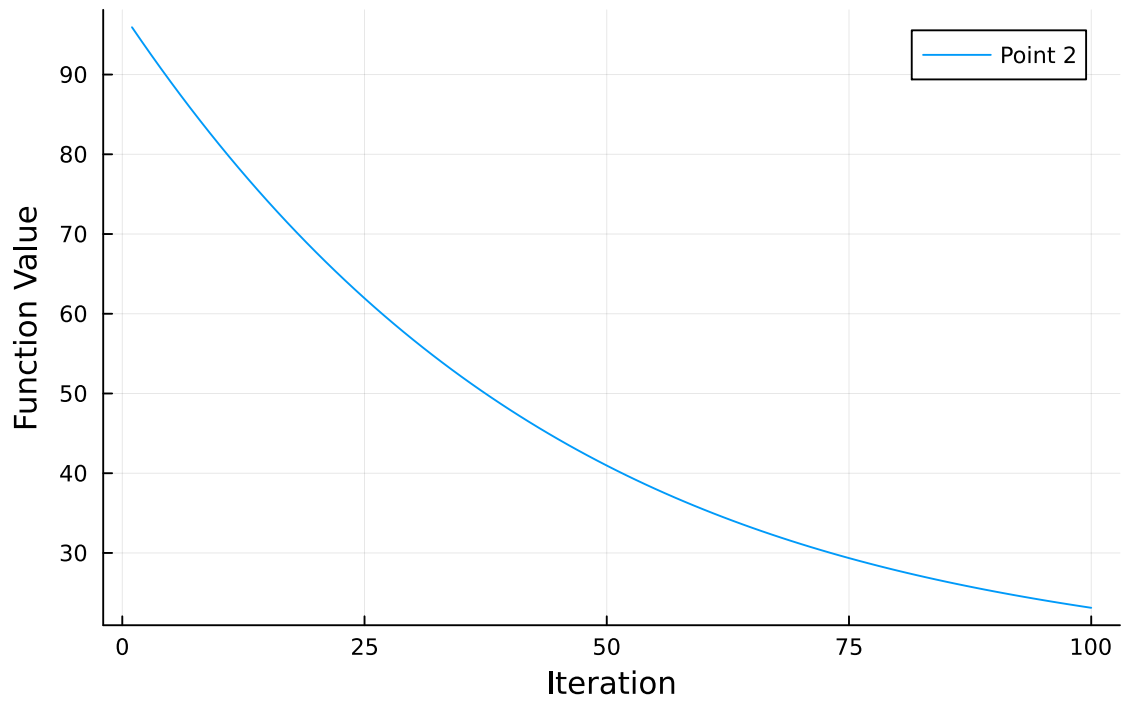
1.5.1 Zbieżność - $\alpha = 0.01$

```
In [7]: for i in 1:10
     $\theta$ , f_values = steepest_gradient_descent(f, f_grad, points[i], 0.01; max_iter
    plot(1:length(f_values), f_values, label="Point  $i$ ")
    display(plot!(title="Optimization from Point  $i$ ", xlabel="Iteration", ylabel
end
```

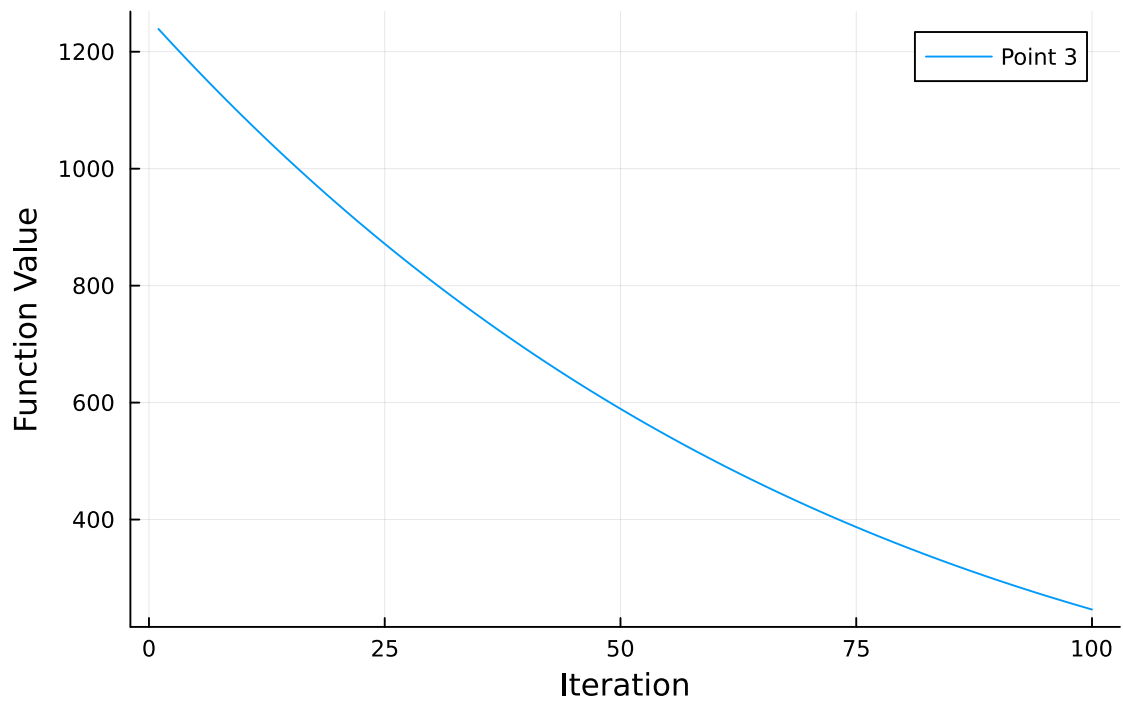
Optimization from Point 1



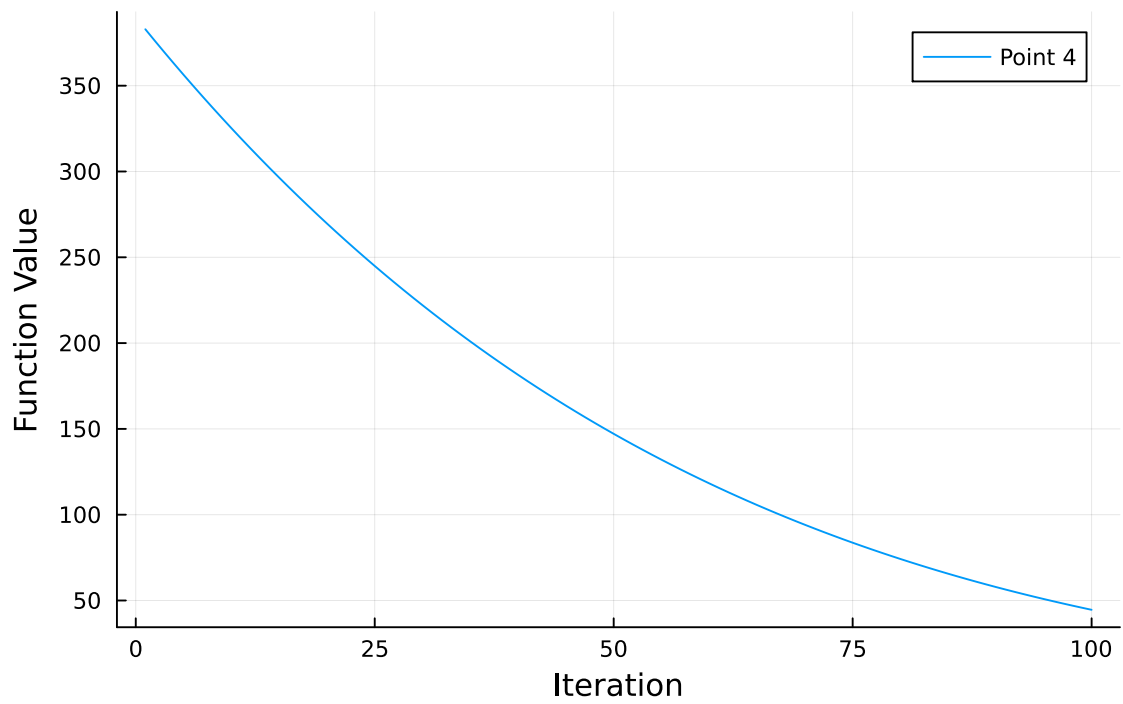
Optimization from Point 2



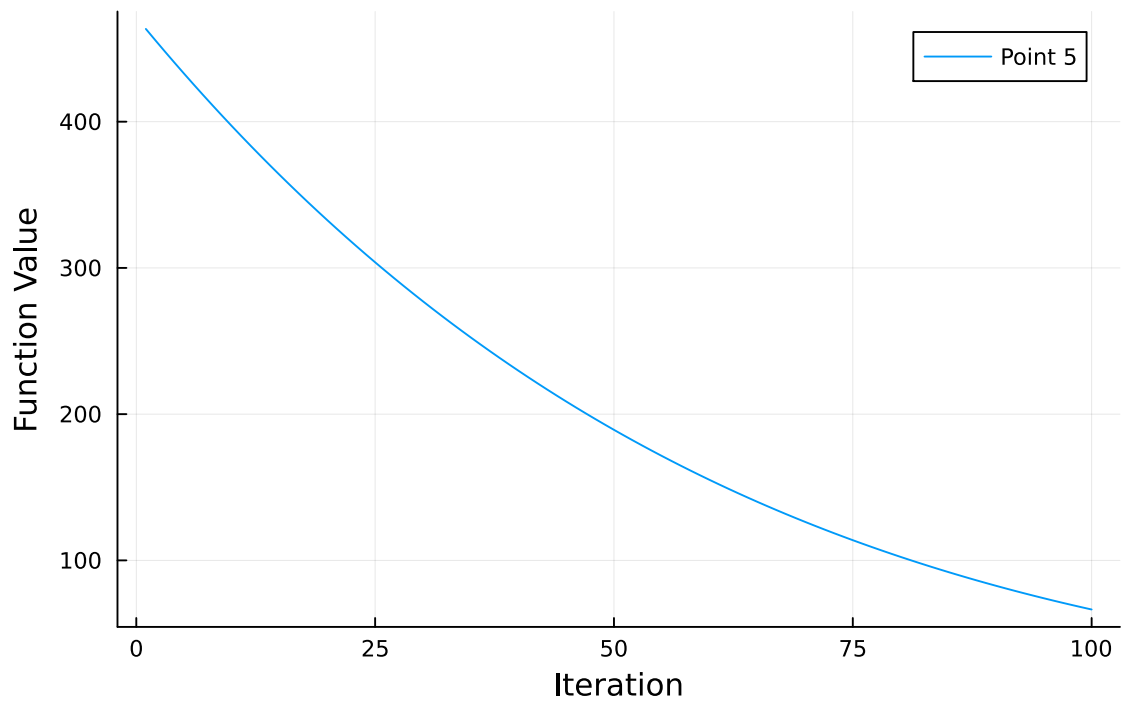
Optimization from Point 3



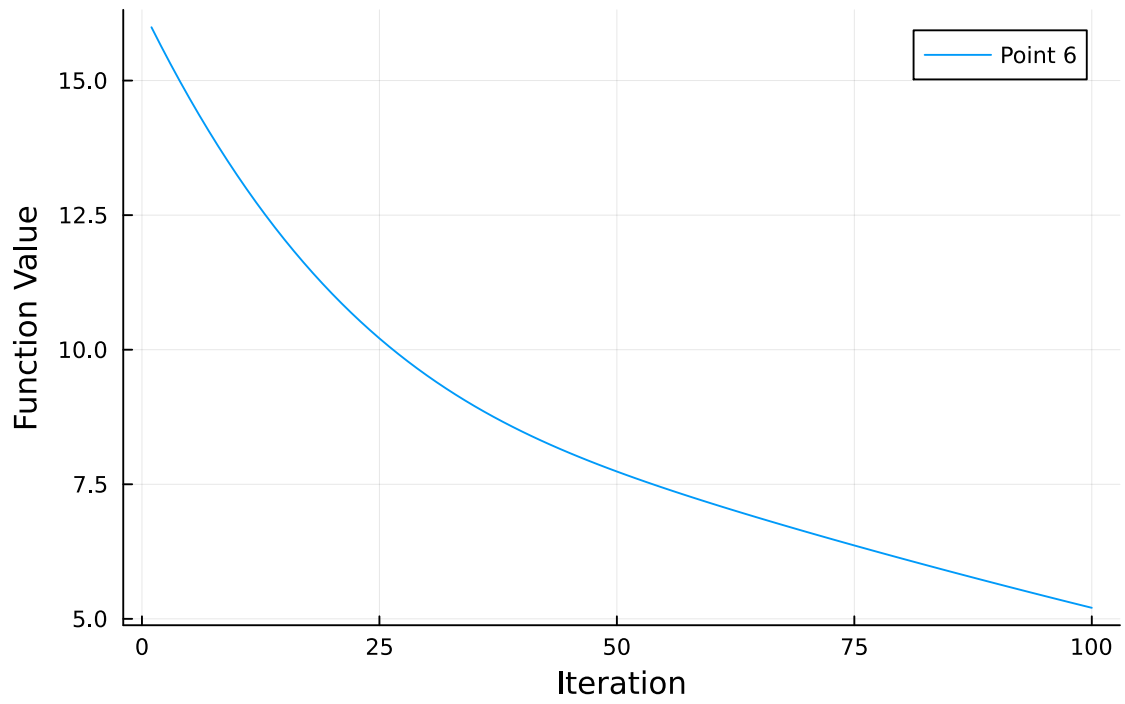
Optimization from Point 4



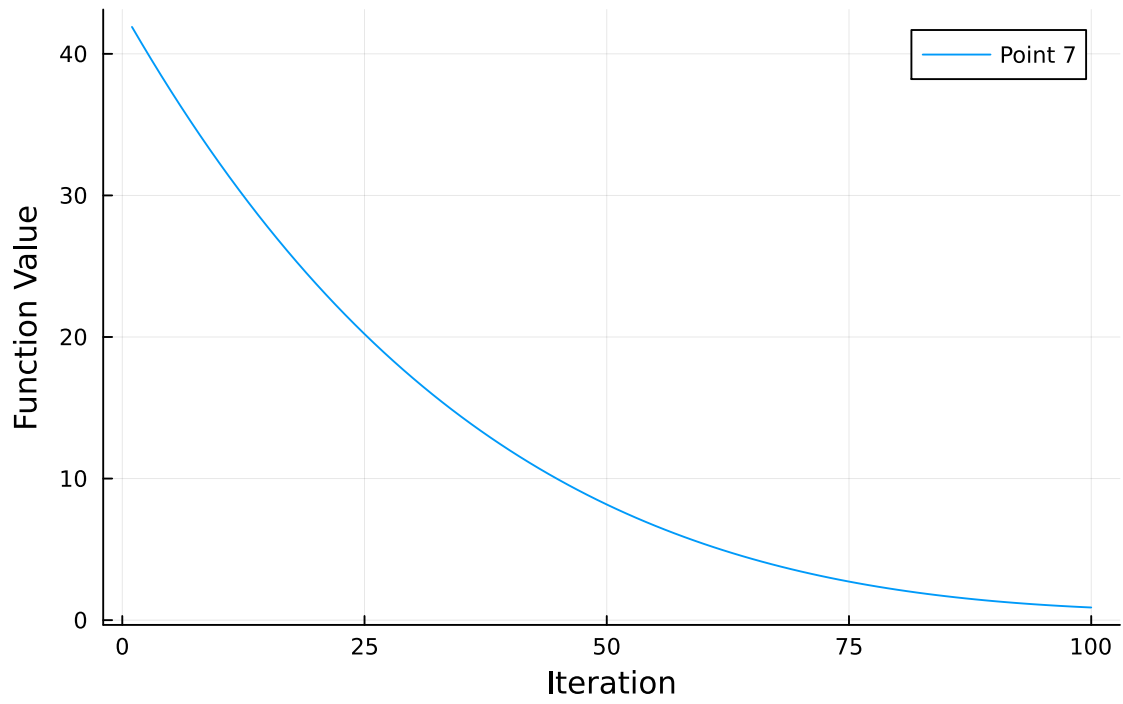
Optimization from Point 5



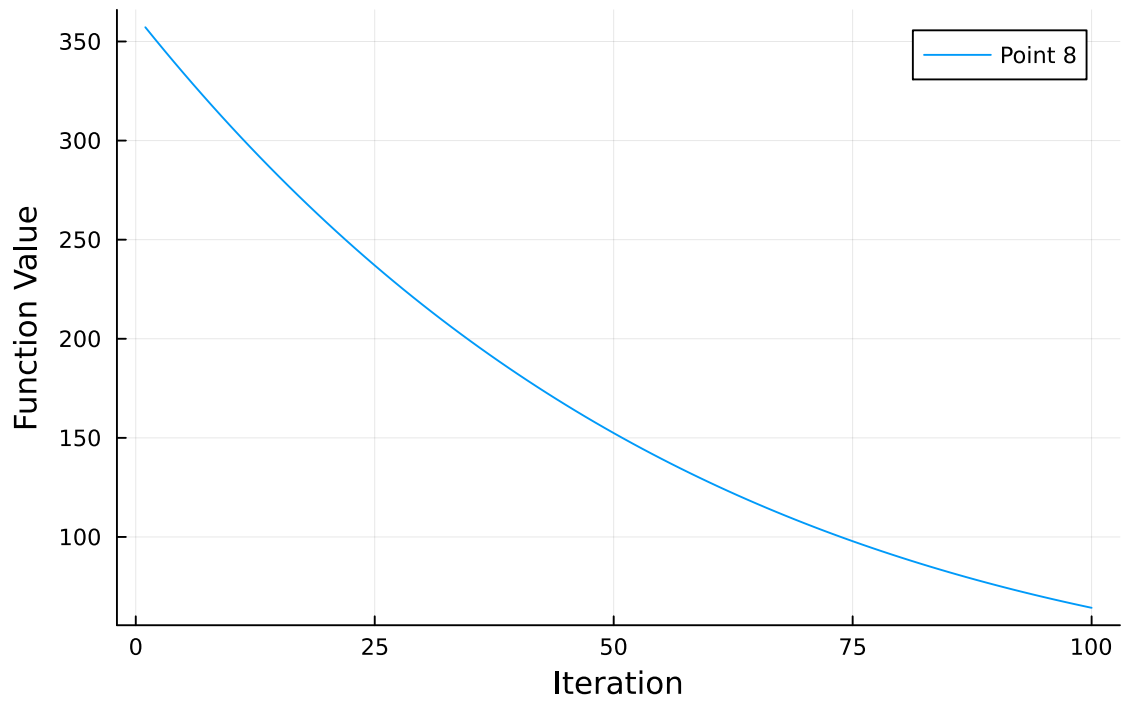
Optimization from Point 6



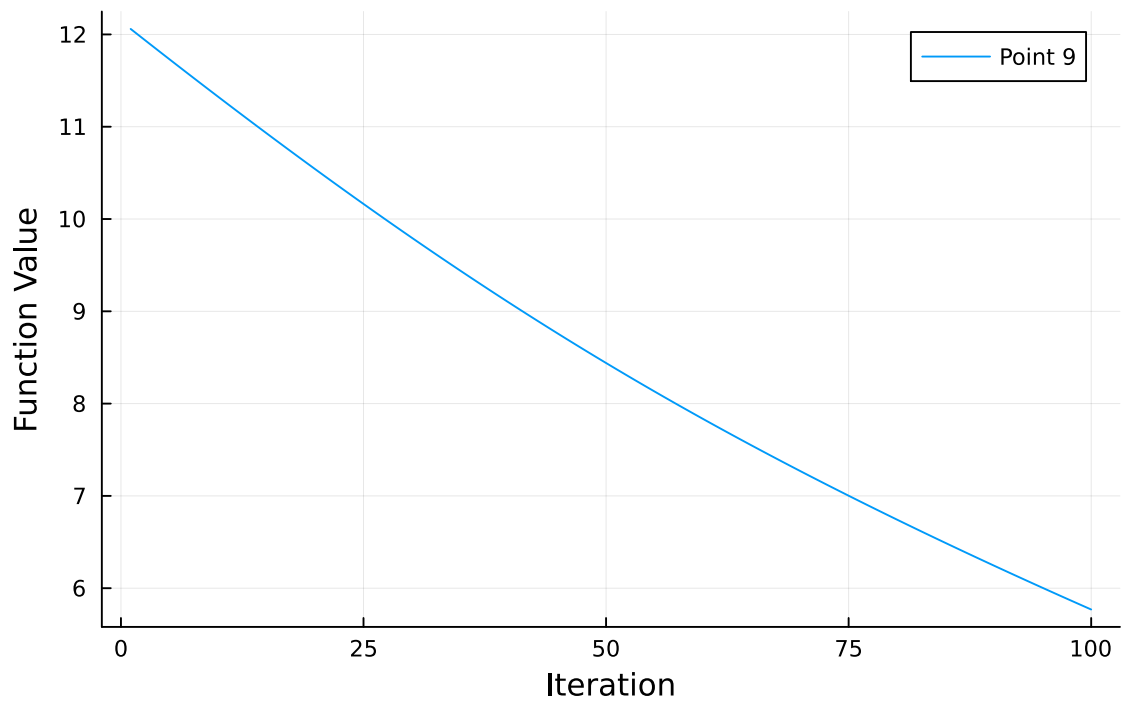
Optimization from Point 7



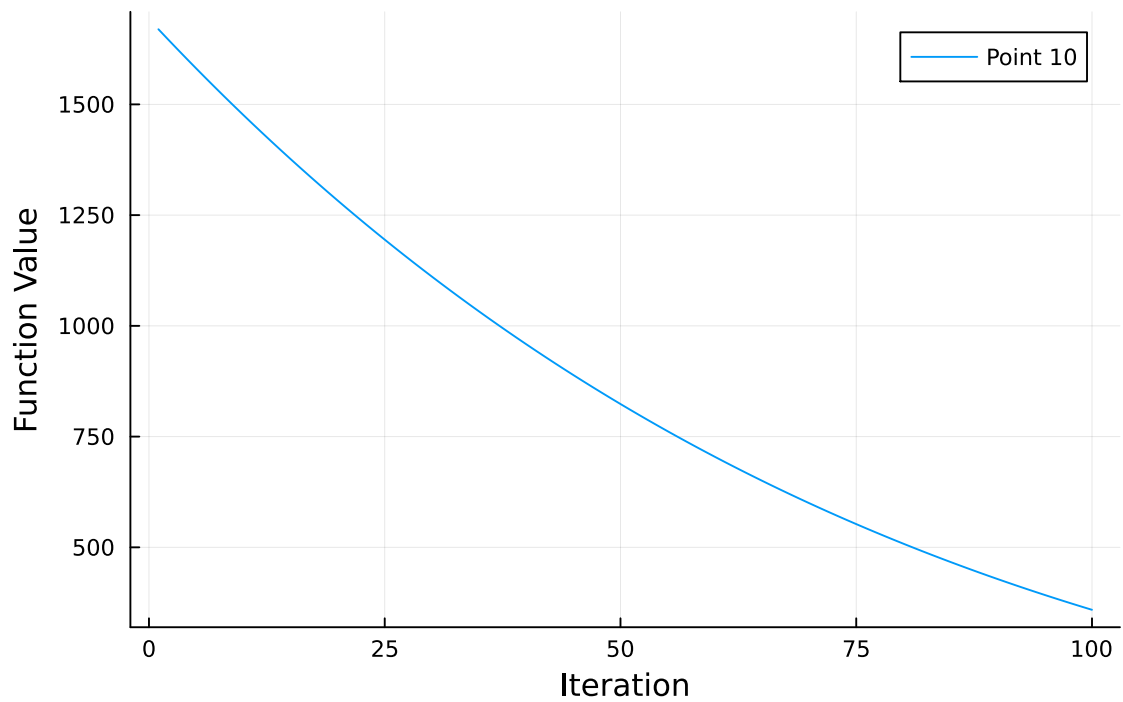
Optimization from Point 8



Optimization from Point 9



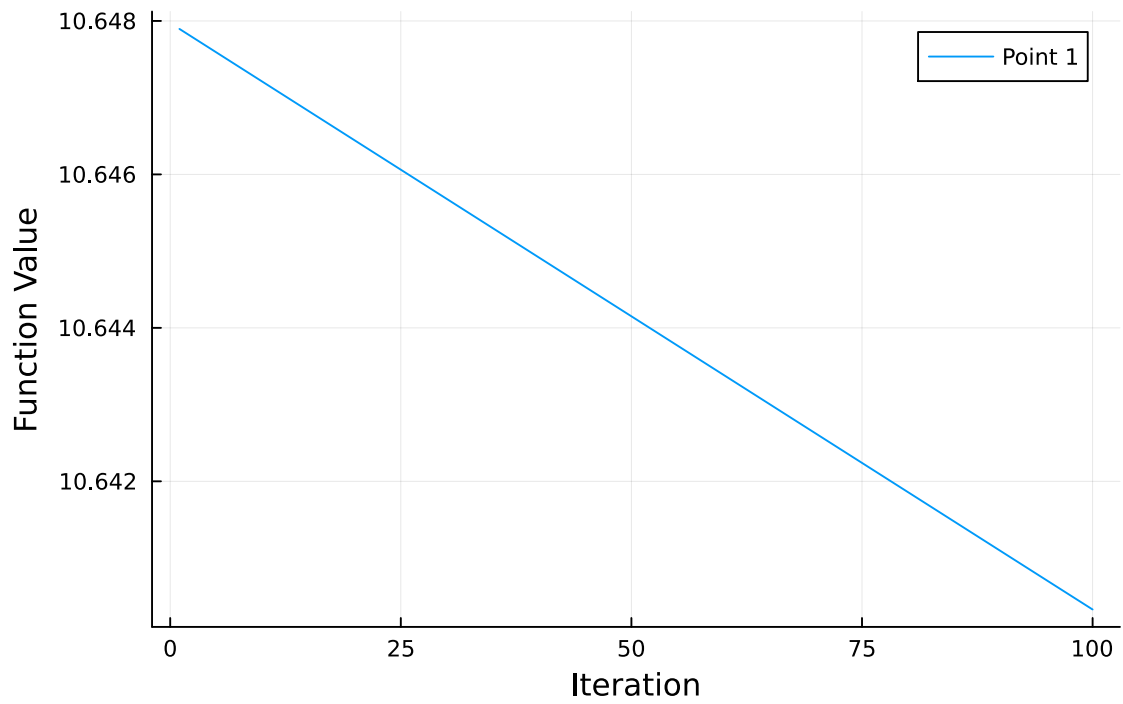
Optimization from Point 10



1.5.2 Powolna zbieżność - $\alpha = 0.00001$

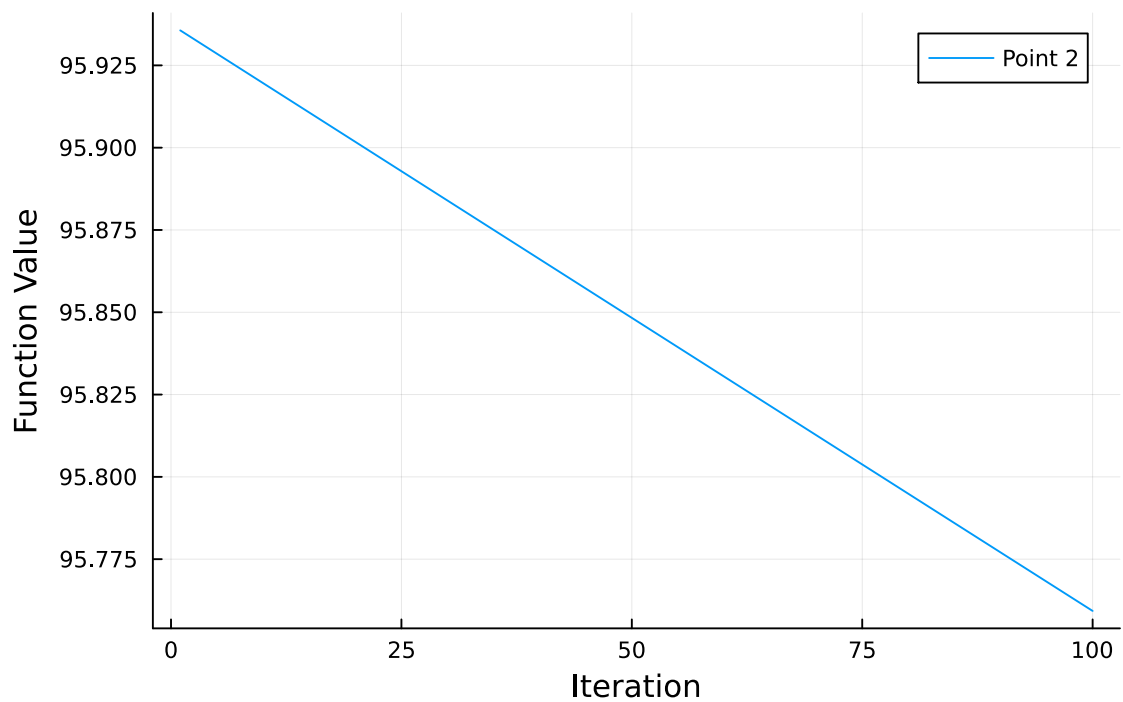
```
In [8]: for i in 1:10
        θ, f_values = steepest_gradient_descent(f, f_grad, points[i], 0.00001; max_i
        plot(1:length(f_values), f_values, label="Point $i")
        display(plot!(title="Optimization from Point $i", xlabel="Iteration", ylabel
        println("Minimum: ", θ)
    end
```

Optimization from Point 1

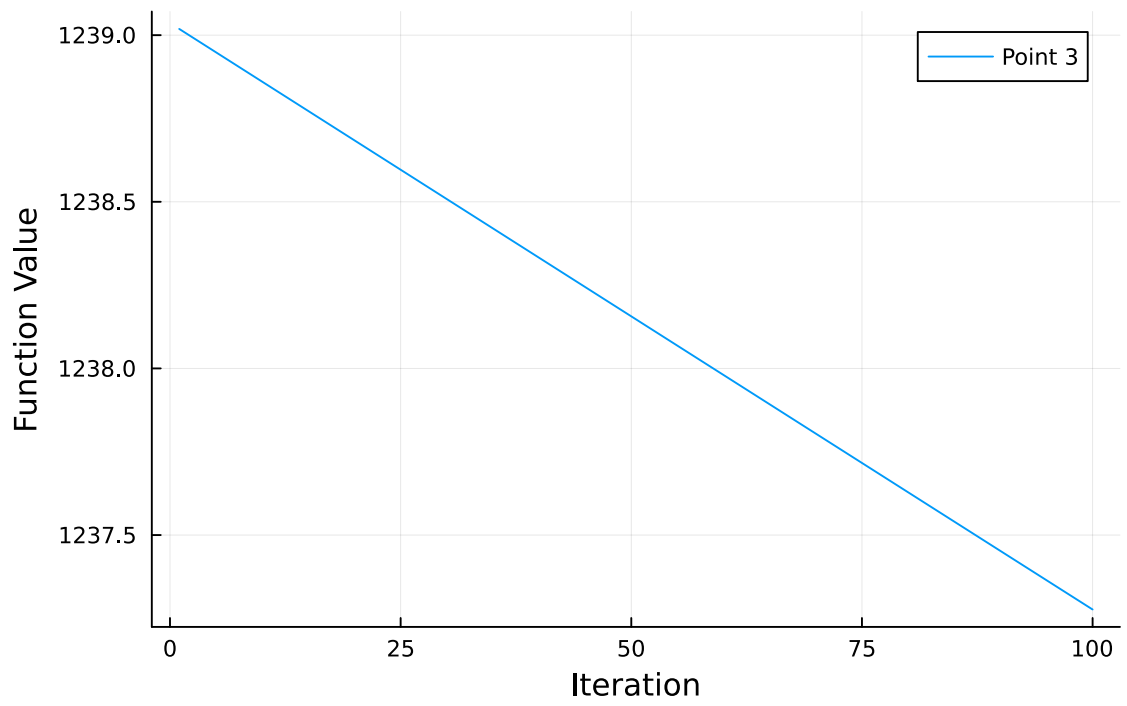


Minimum: [0.23902590135540047, 3.1275882117427316]
Minimum: [3.1010654824593313, 3.7228854305641277]
Minimum: [4.643462416310732, 1.7980925516335995]
Minimum: [-3.9168383138928884, 2.7914142114972456]
Minimum: [4.017336050631339, -4.708706692388488]
Minimum: [2.4693008638702847, -3.7921044896617557]
Minimum: [2.918911829306383, -1.5839153506218195]
Minimum: [-3.817183208105785, -4.291085096209277]
Minimum: [-1.5047514320687425, 4.149064367923264]
Minimum: [-4.863320648834719, -0.43383007538633]

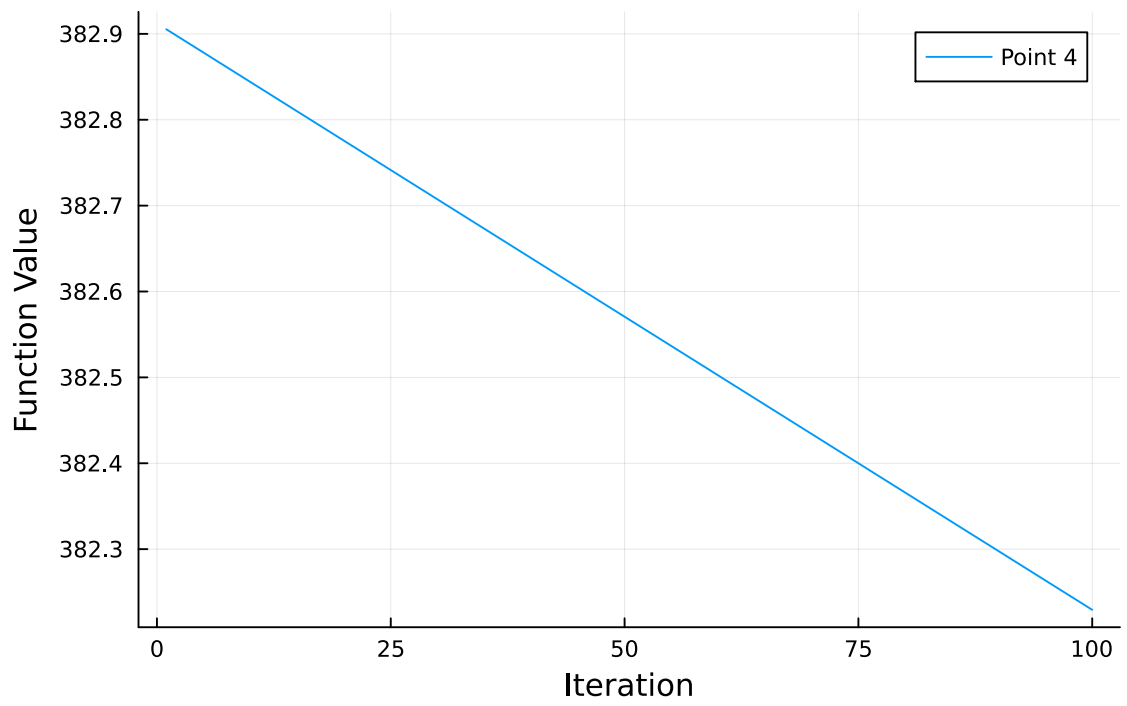
Optimization from Point 2



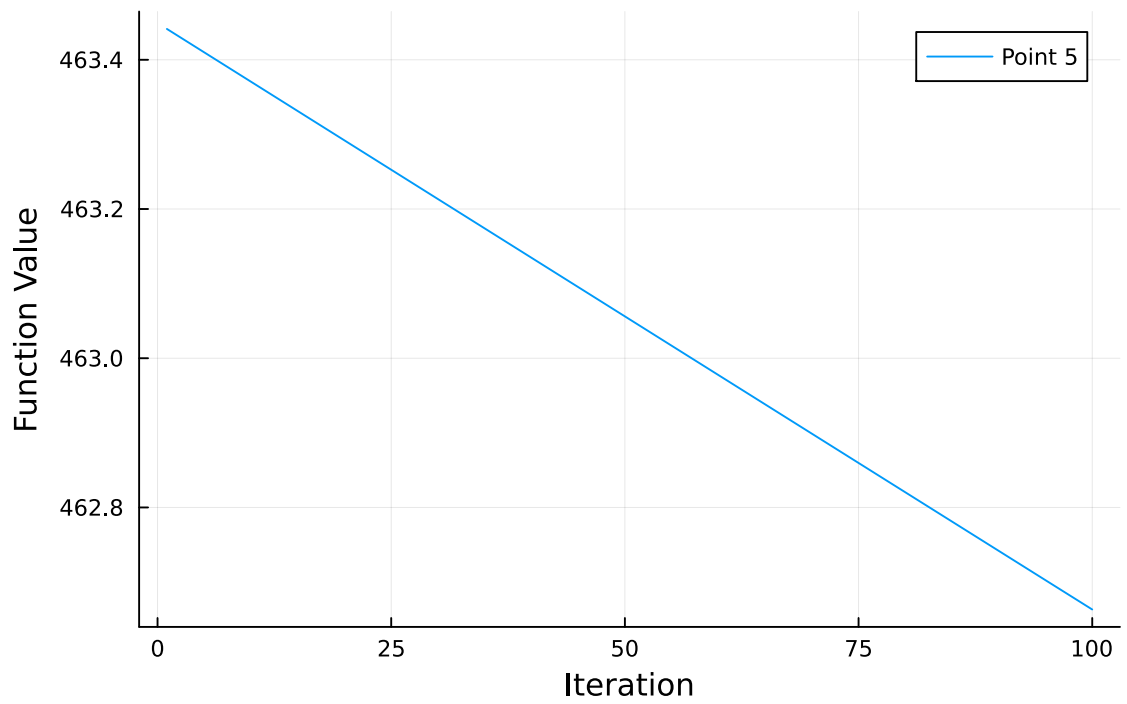
Optimization from Point 3



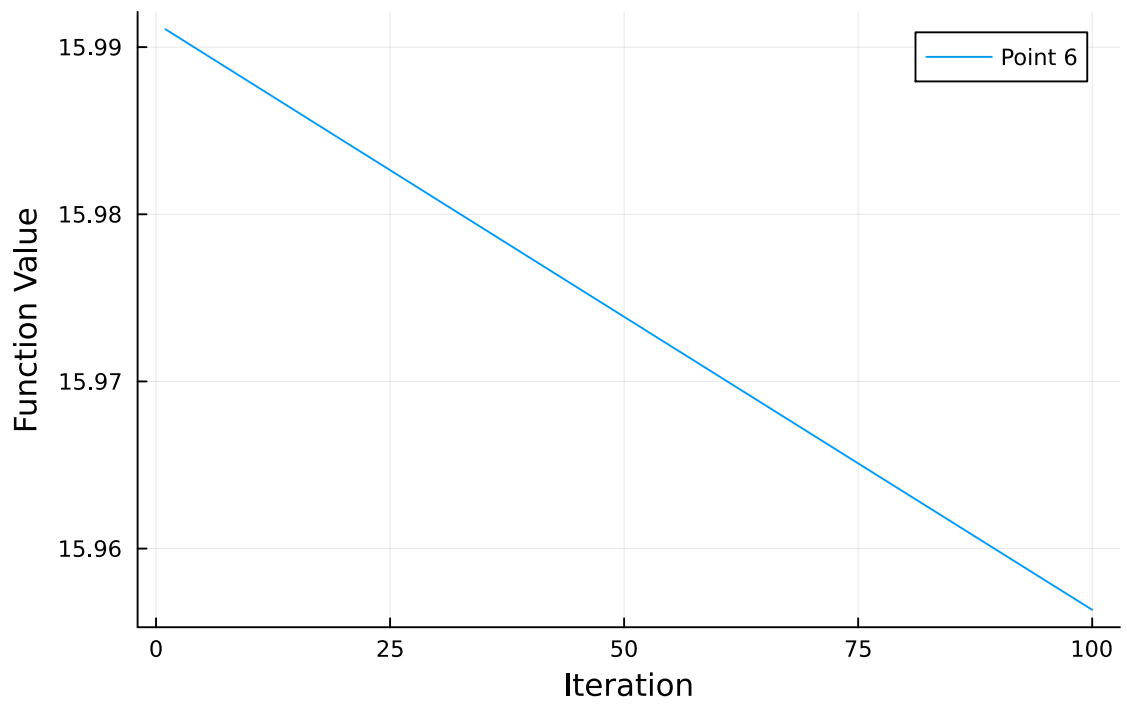
Optimization from Point 4



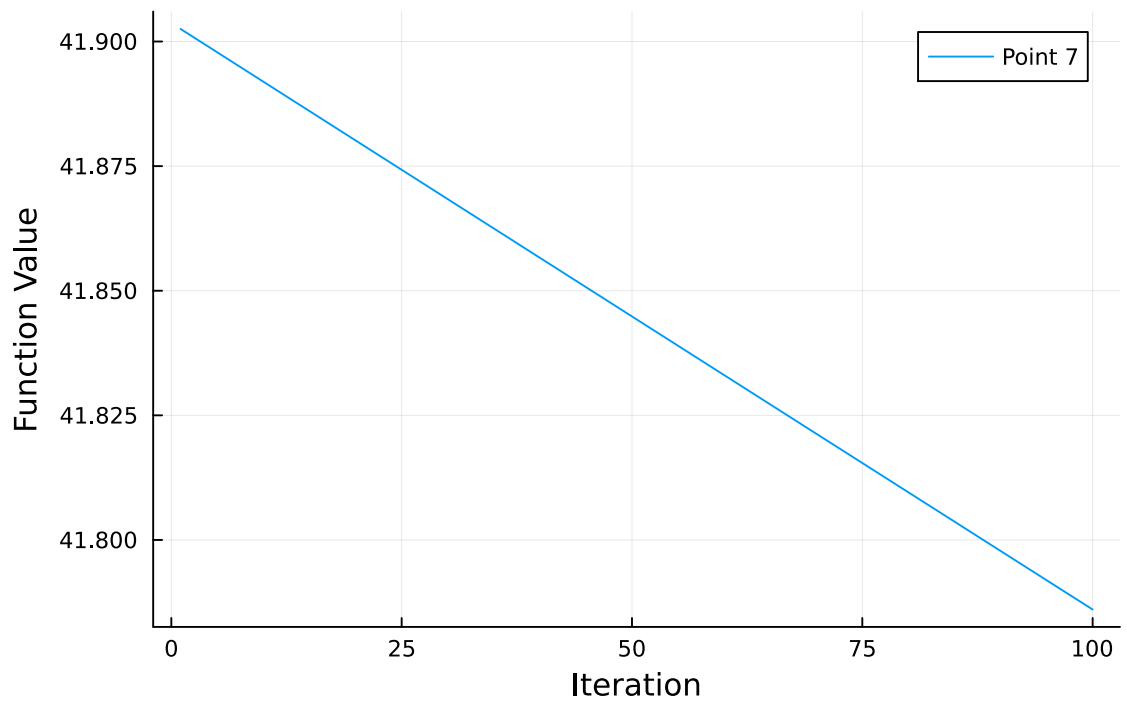
Optimization from Point 5



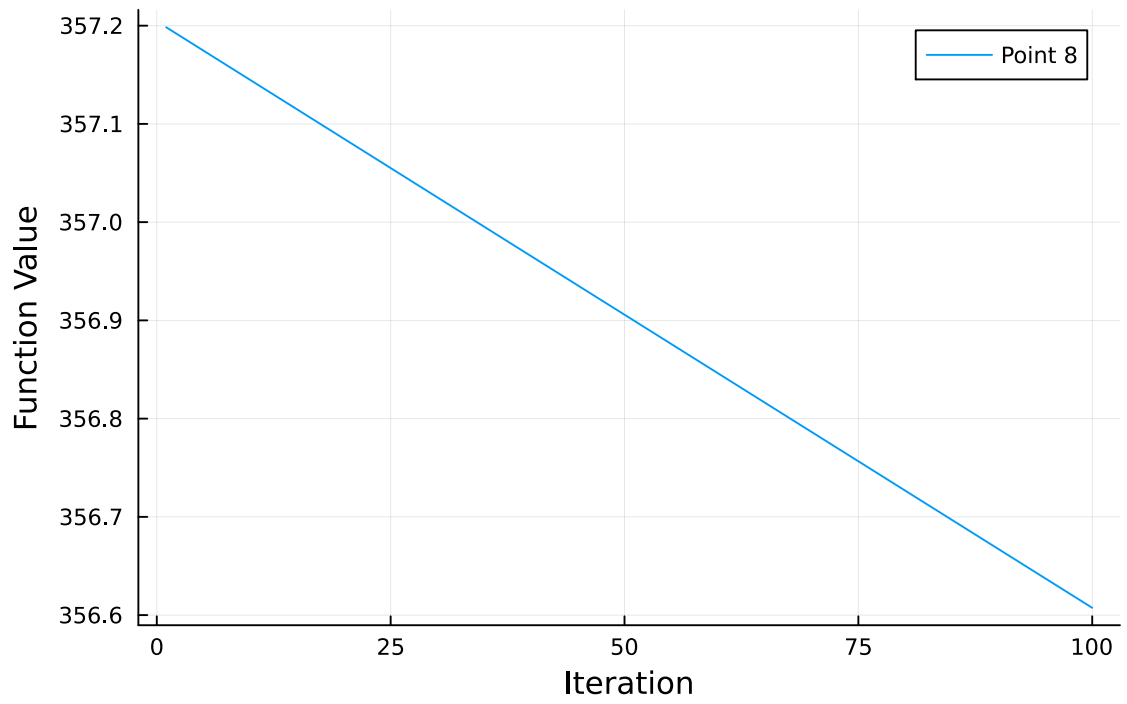
Optimization from Point 6

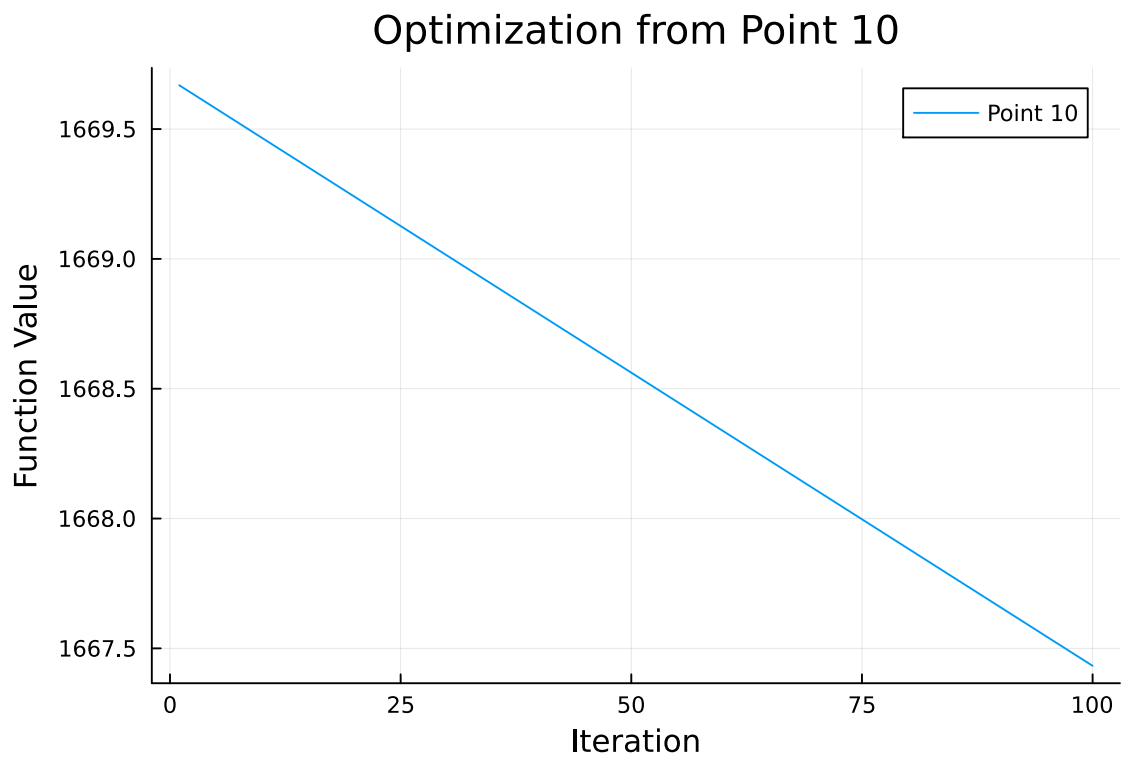
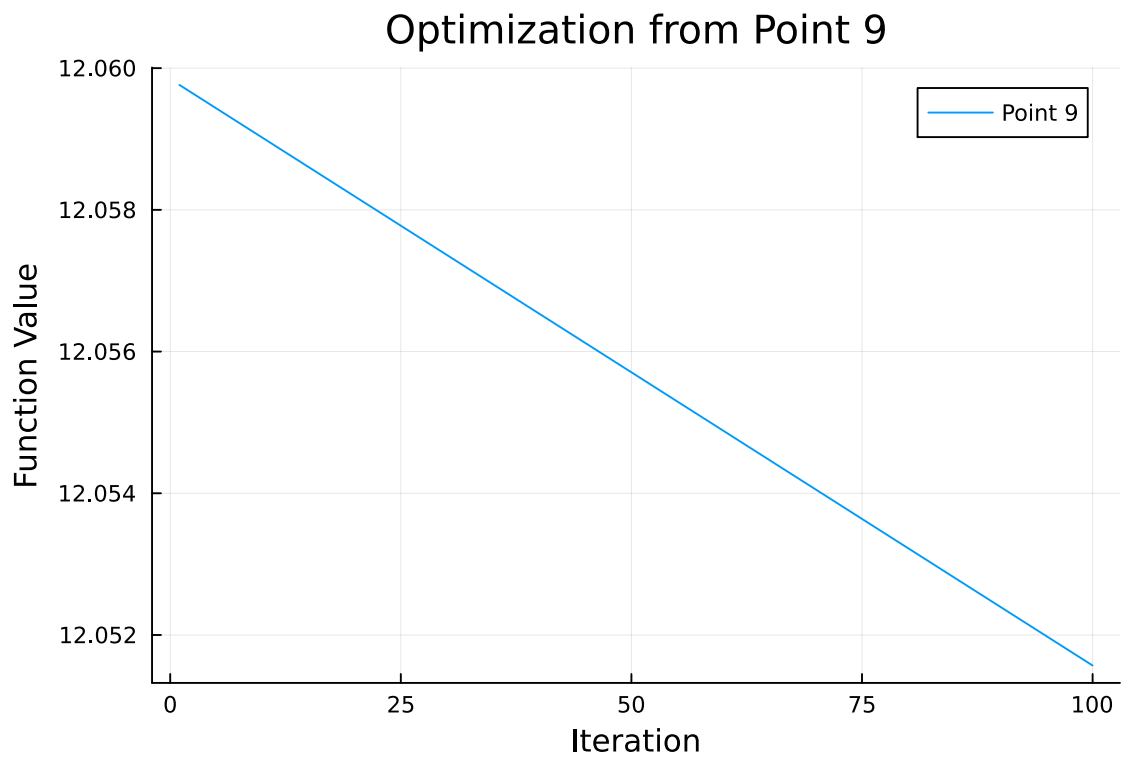


Optimization from Point 7



Optimization from Point 8

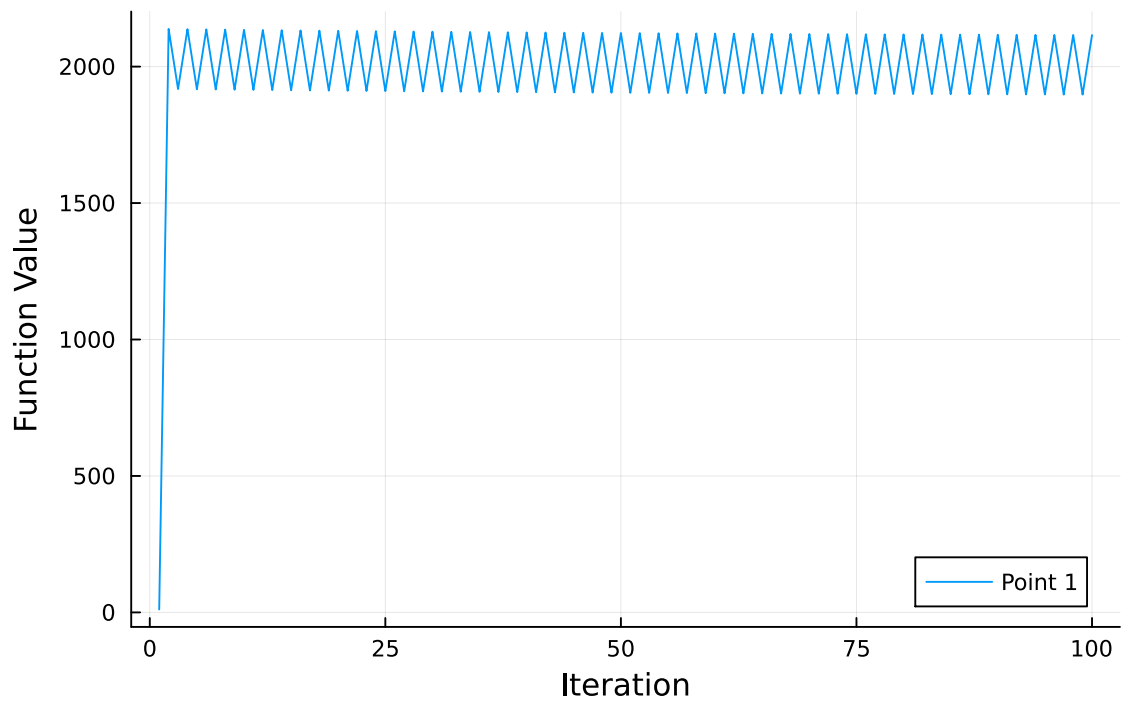




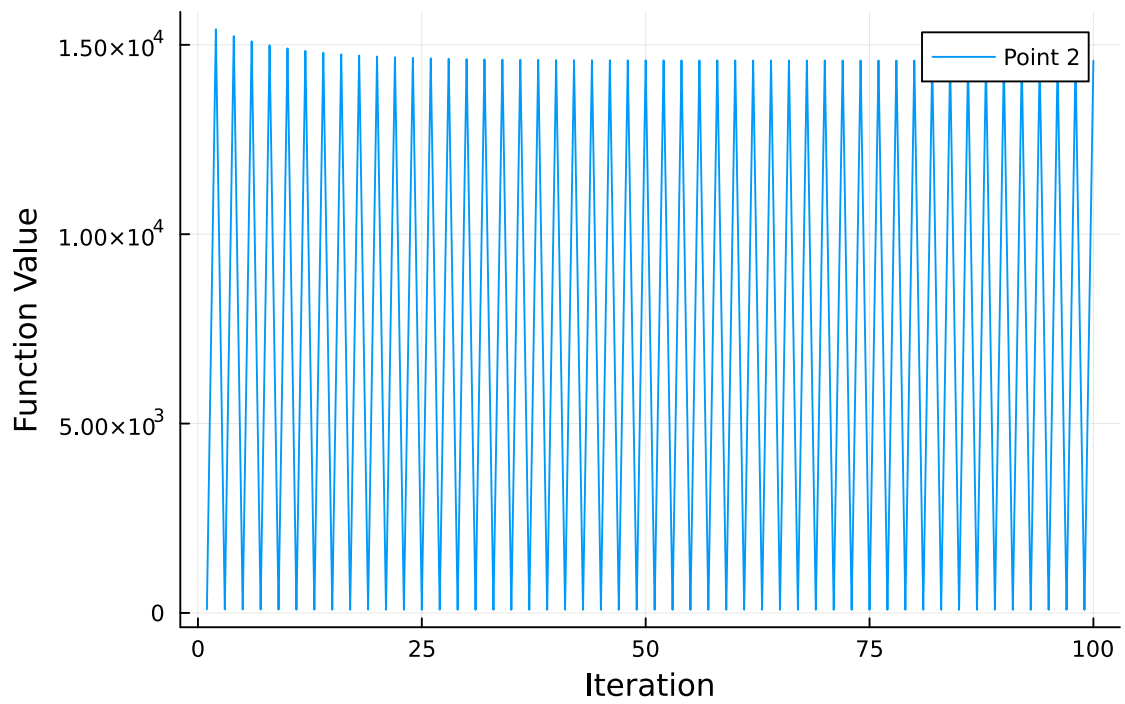
1.5.3 Brak zbieżności - $\alpha = 10$

```
In [9]: for i in 1:10
        θ, f_values = steepest_gradient_descent(f, f_grad, points[i], 10; max_iter=1
        plot(1:length(f_values), f_values, label="Point $i")
        display(plot!(title="Optimization from Point $i", xlabel="Iteration", ylabel
        end
```

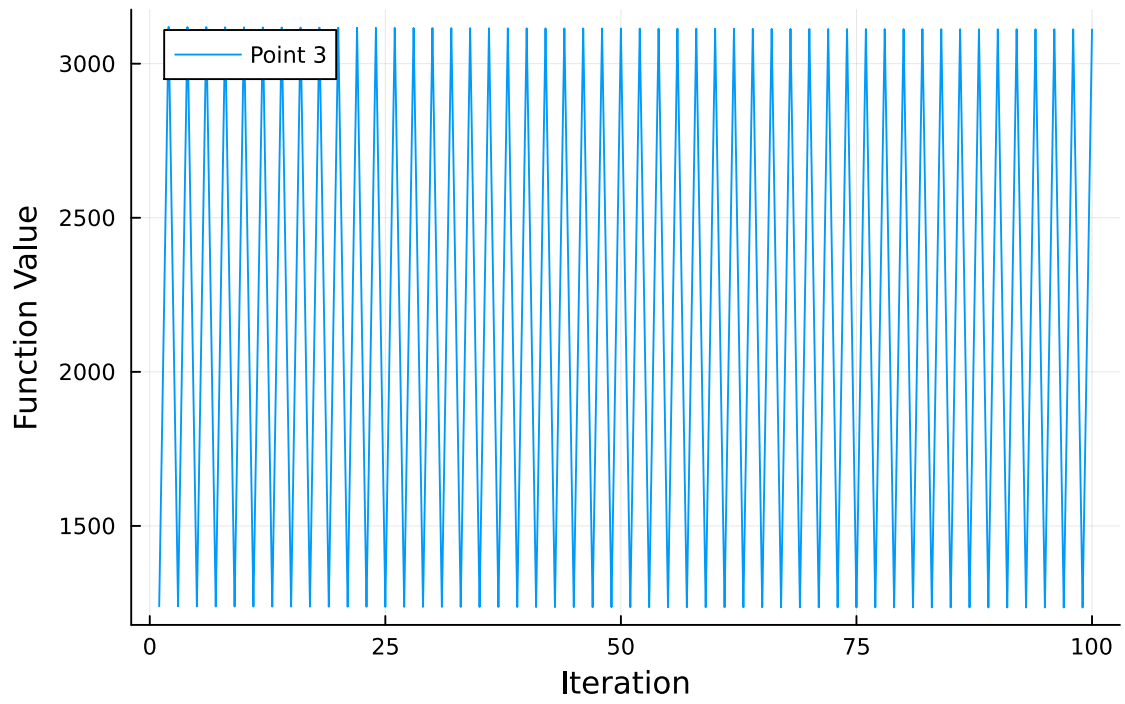
Optimization from Point 1



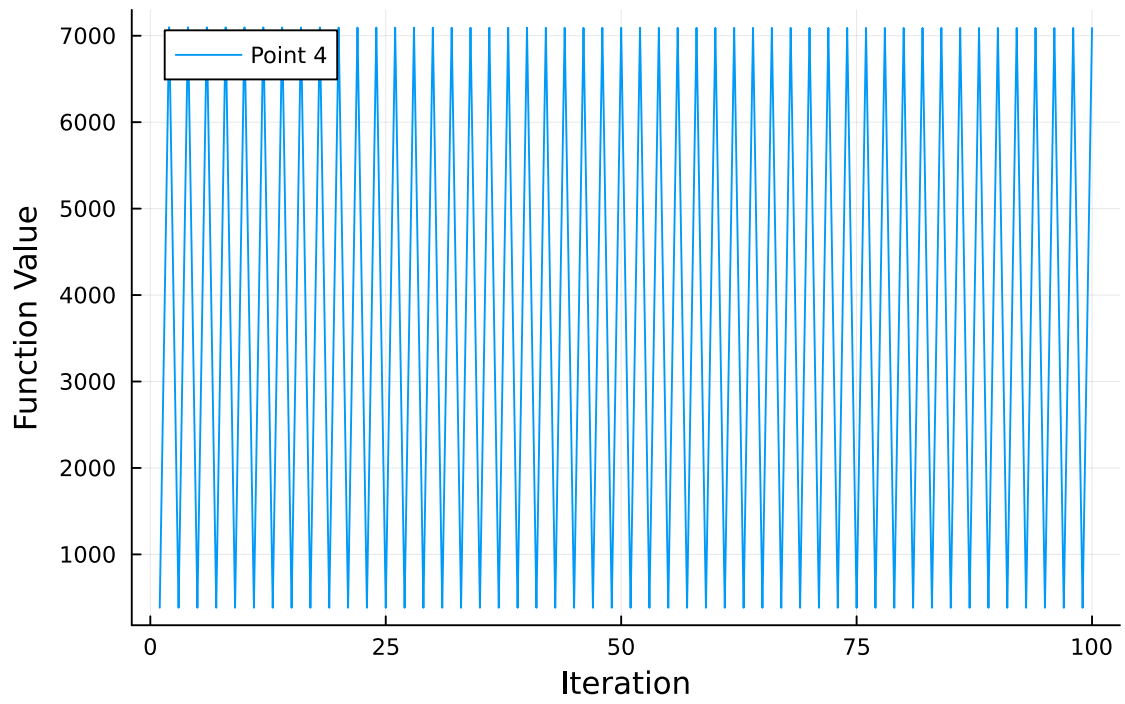
Optimization from Point 2



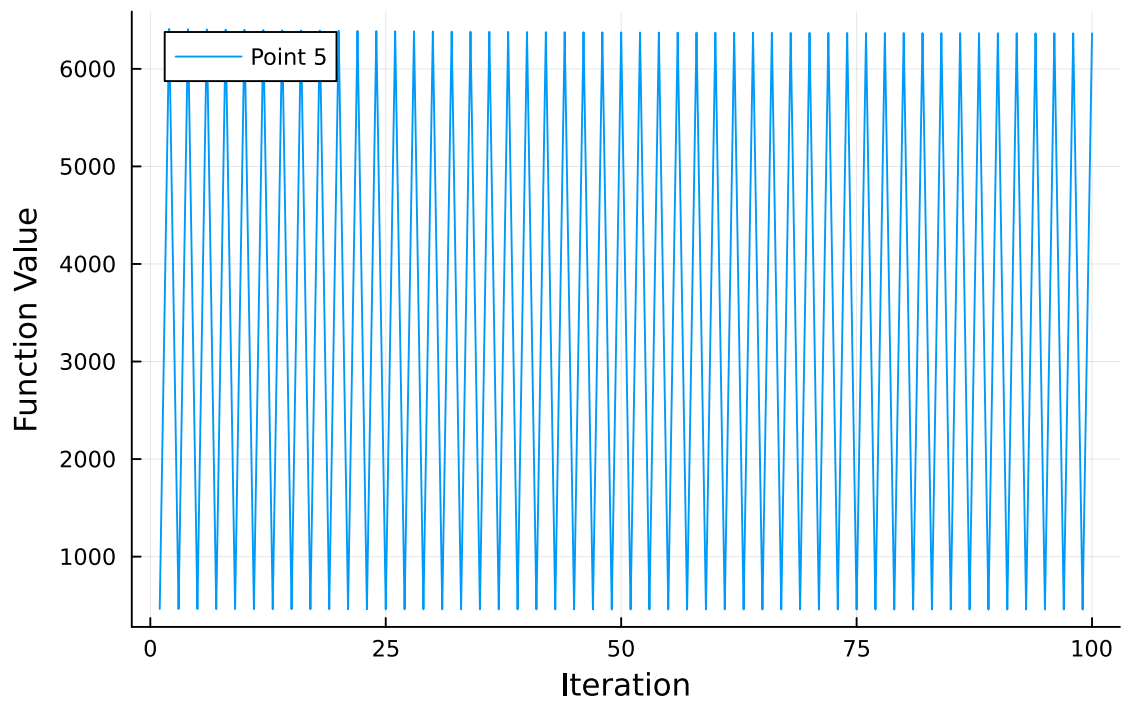
Optimization from Point 3



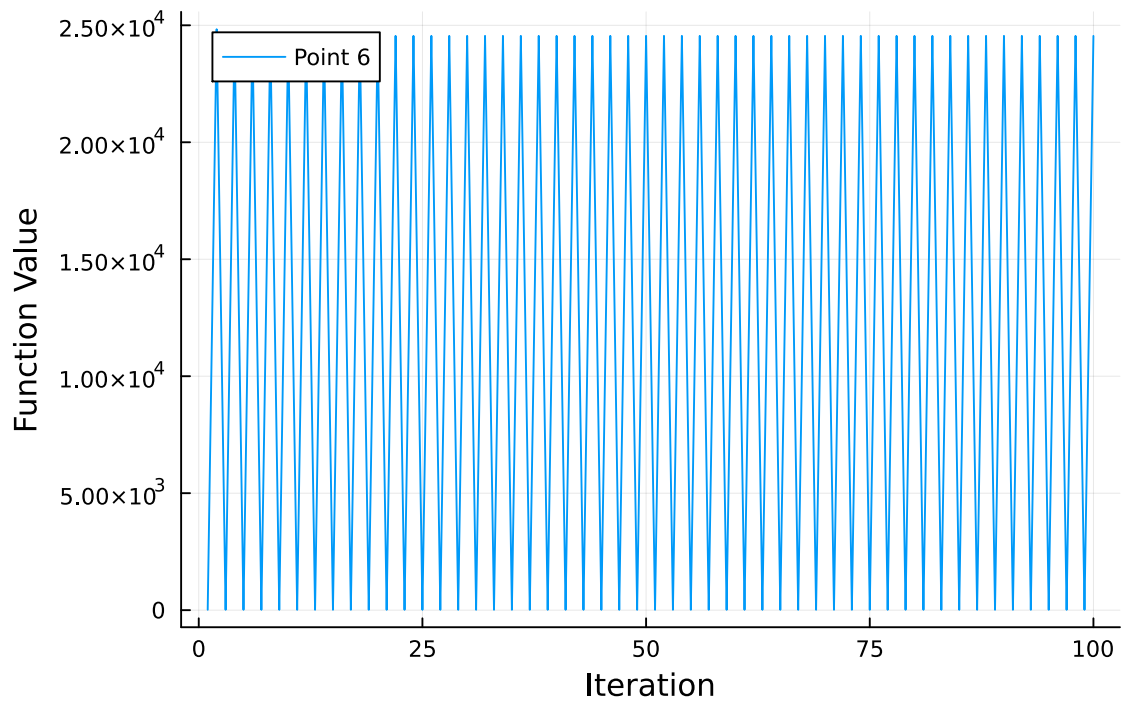
Optimization from Point 4



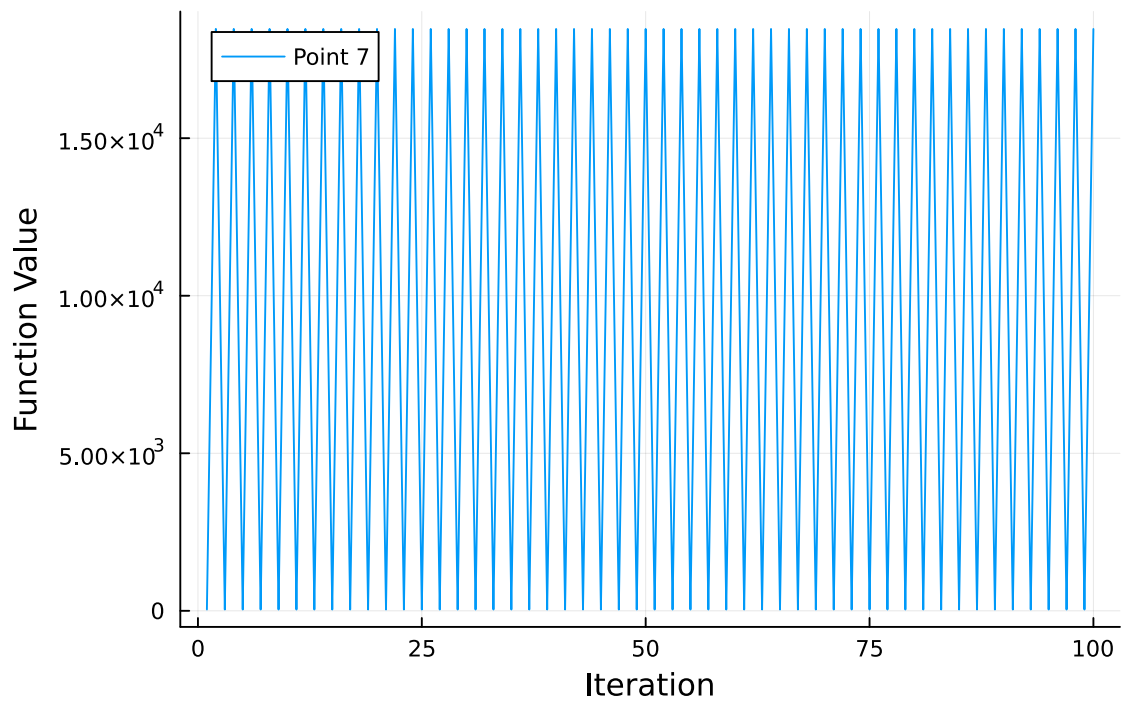
Optimization from Point 5



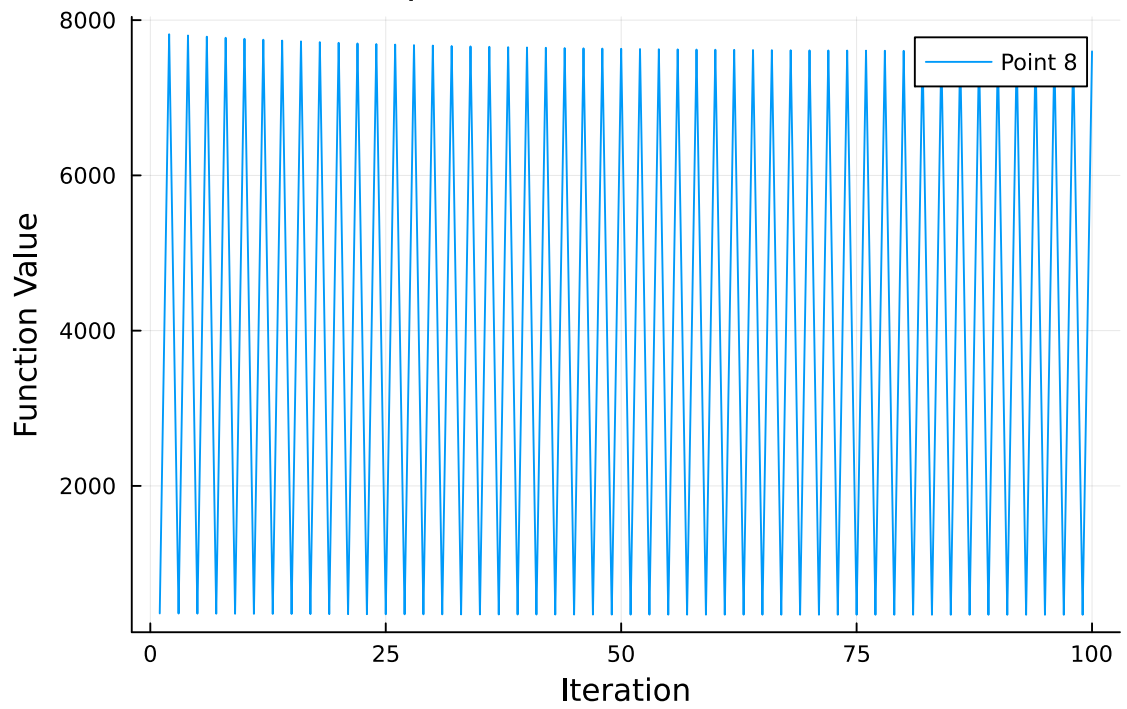
Optimization from Point 6



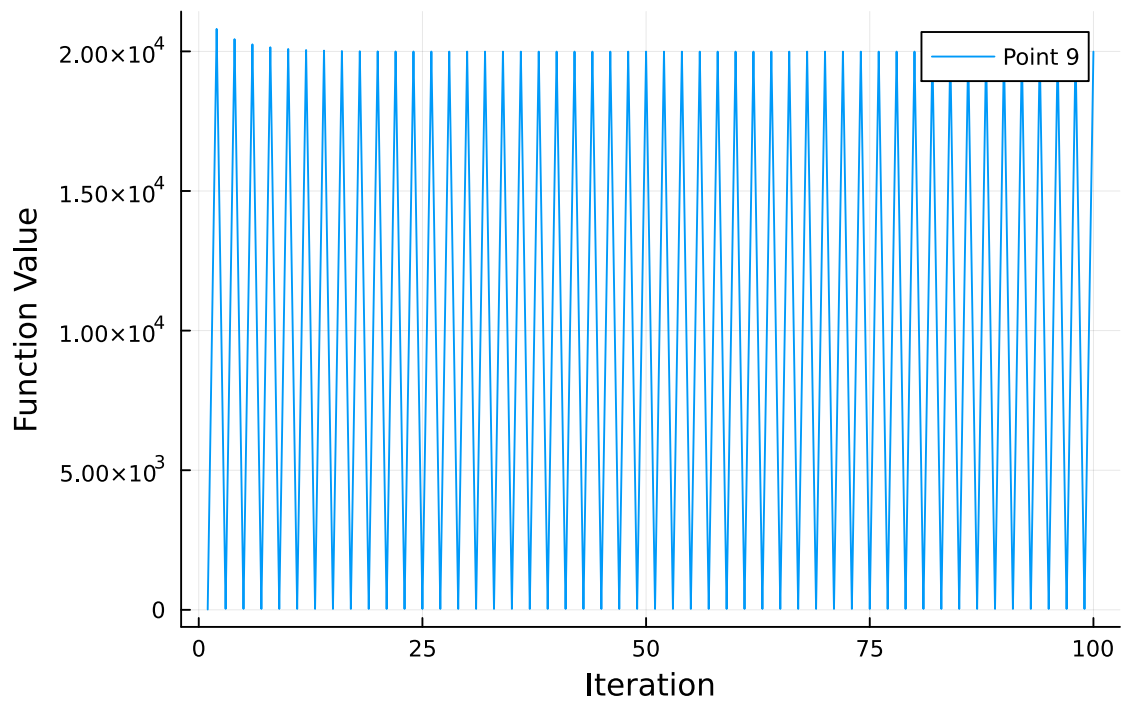
Optimization from Point 7



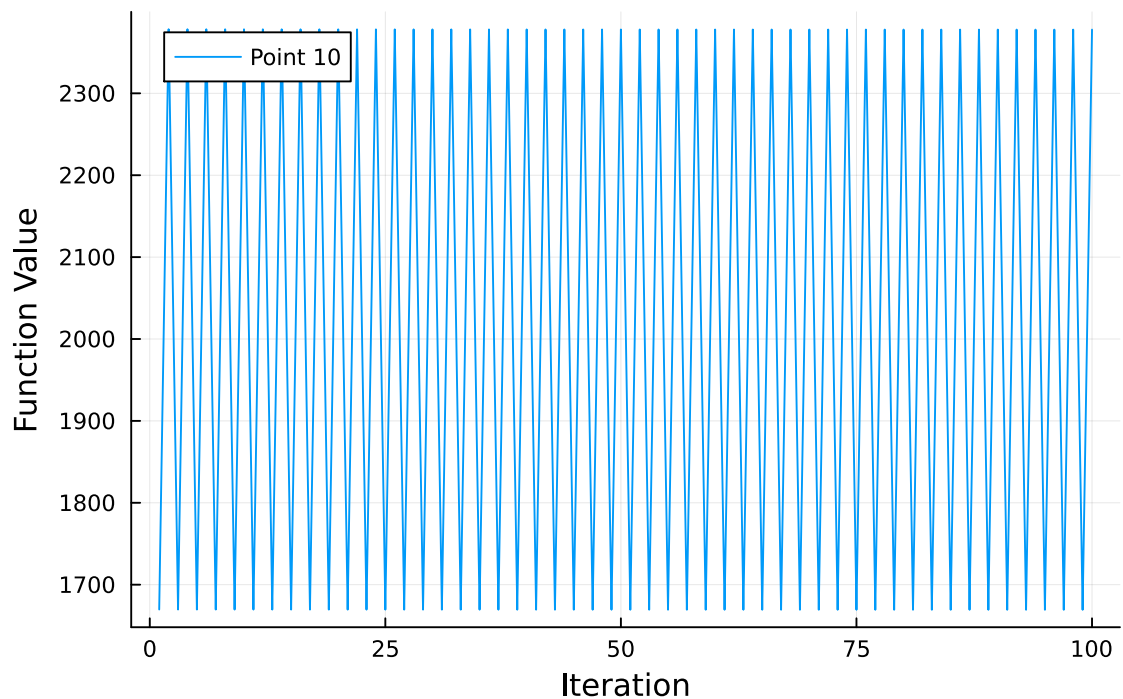
Optimization from Point 8



Optimization from Point 9



Optimization from Point 10



1.6 Armijo

`steepest_gradient_descent_armijo(cost, grad, x0,  $\alpha$ , p;  $\beta=1e-4$ , max_iter=100, tol=1e-6)` - Zamiana stałego kroku na warunek wystarczającego spadku. Funkcja działa podobnie jak metoda największego spadku z tą różnicą, że w przypadku zbyt niskiej wartości gradientu (funkcja maleje bardzo wolno, jest niemal stała) powiększany jest rozmiar kroku α , żeby szybciej znajdowała minimum globalne (szybsza optymalizacja funkcji kosztu).

Argumenty:

- cost - funkcja kosztu, funkcja f minimalizowana
- grad - funkcja obliczająca pochodne w różnych kierunkach
- x0 - wektor wartości współrzędnych początkowych punktu
- α - rozmiar kroku
- p - współczynnik powiększający krok w warunku Armijo
- β - współczynnik w warunku Armijo
- max_iter - maksymalna liczba iteracji
- tol- współczynnik tolerancji

```
In [10]: function steepest_gradient_descent_armijo(cost, grad, x0,  $\alpha$ , p;  $\beta=1e-4$ , max_iter)
     $\theta$  = copy(x0)
    f_values = zeros(max_iter)
    storage = zeros(length( $\theta$ ))
    for i in 1:max_iter
        value_start = cost( $\theta$ )
        storage = grad( $\theta$ )
        norm_storage = norm(storage)
        if norm_storage == 0
            break
        end
         $\alpha_2$  = copy( $\alpha$ )
        while cost( $\theta$  -  $\alpha_2$  * storage) > value_start -  $\beta$  *  $\alpha_2$  * dot(storage, sto
             $\alpha_2$  *= p
        end
         $\theta_{new}$  =  $\theta$  - storage .*  $\alpha_2$ 
        value_stop = cost( $\theta_{new}$ )
        f_values[i] = value_start
        if abs(value_stop - value_start) < tol
            break
        else
             $\theta$  =  $\theta_{new}$ 
        end
    end
    return  $\theta$ , f_values
end
```

steepest_gradient_descent_armijo (generic function with 1 method)

1.7 Przeszukiwanie liniowe - funkcja optymalizująca

linear_search(cost, grad, x0, div; max_iter=100, tol=1e-6) - Funkcja ta korzysta z bracketu [0, 1] z wycinka w określonym kierunku. Odcinek jest dzielony na określoną przez parametr div liczbę punktów i wybiera ten, dla którego wartość funkcji jest najmniejsza.

Argumenty:

- cost - funkcja f
- grad - gradienty funkcji f
- x0 - wektor współrzędnych początkowych
- div - liczba punktów dzielących bracket [0, 1] z przekroju funkcji w określonych przez obliczony gradient kierunku

- max_iter - maksymalna ilość iteracji funkcji
- tol - współczynnik tolerancji

```
In [11]: function linear_search(cost, grad, x0, div; max_iter=100, tol=1e-6)
    θ = copy(x0)
    f_values = zeros(max_iter)
    storage = zeros(length(θ))
    for i in 1:max_iter
        value_start = cost(θ)
        storage = grad(θ)
        norm_storage = norm(storage)
        θ_min = typemax(Float64)
        if norm_storage == 0
            break
        end
        θ_new = θ - storage .* (1 / norm_storage)
        for j in 0:div-1
            θ_x = θ + (θ_new - θ) * ((j + 0.5) / div)
            value_x = cost(θ_x)
            if value_x < θ_min
                θ_min = value_x
                θ = θ_x
            end
        end
        value_stop = cost(θ_new)
        f_values[i] = value_start
        if abs(value_stop - value_start) < tol
            break
        else
            θ = θ_new
        end
    end
    return θ, f_values
end
```

linear_search (generic function with 1 method)

1.8 Porównanie Armijo z przeszukiwaniem liniowym

plot_optimization_methods(α , p) - funkcja rysuje wykresy średnich wartości funkcji dla 10 różnych początkowych punktów w kolejnych iteracjach algorytmów. Funkcja przyjmuje argumenty: α i p, które są ustawiane dla optymalizacji z warunkiem wystarczającego spadku Armijo

```
In [12]: function plot_optimization_methods( $\alpha$ , p)
    m_armijo = zeros(10)
    m_linear = zeros(10)
    for i in 1:10
        θ_1, f_armijo = steepest_gradient_descent_armijo(f, f_grad, points[i],  $\alpha$ , p)
        θ_2, f_linear = linear_search(f, f_grad, points[i], 10; max_iter=10)
        m_armijo += f_armijo
        m_linear += f_linear
    end
    m_armijo ./= 10
```

```

m_linear ./= 10
plot(1:length(m_armijo), m_armijo, label="Armijo Rule")
display(plot!(1:length(m_linear), m_linear, label="Linear Search"))
end

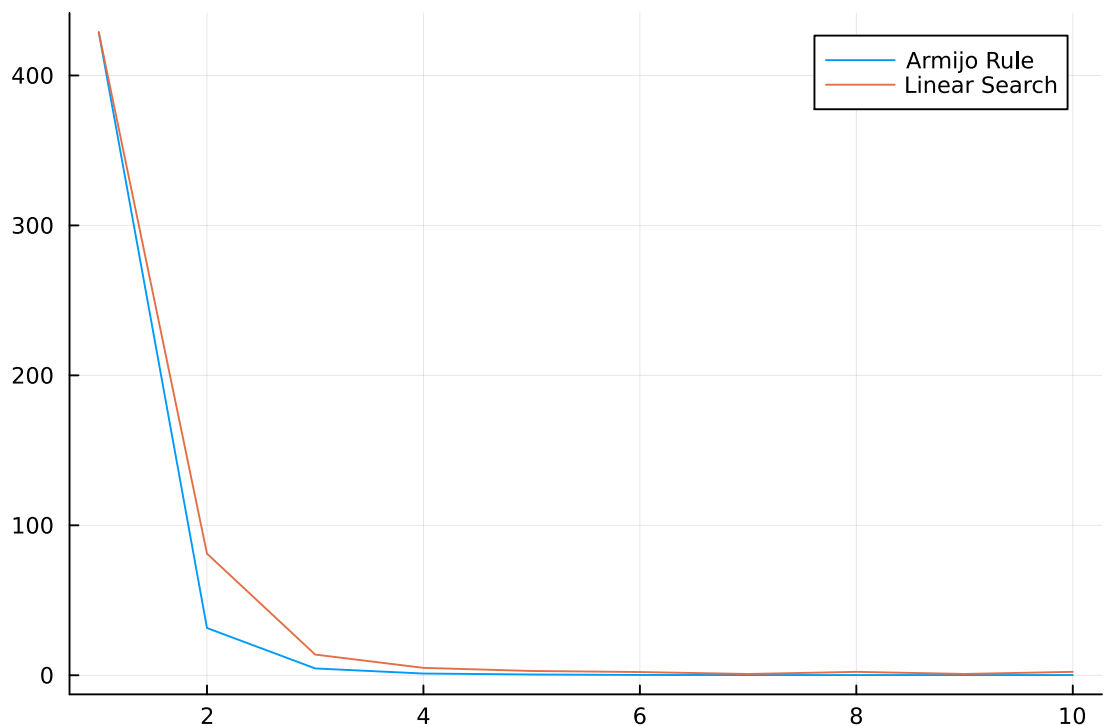
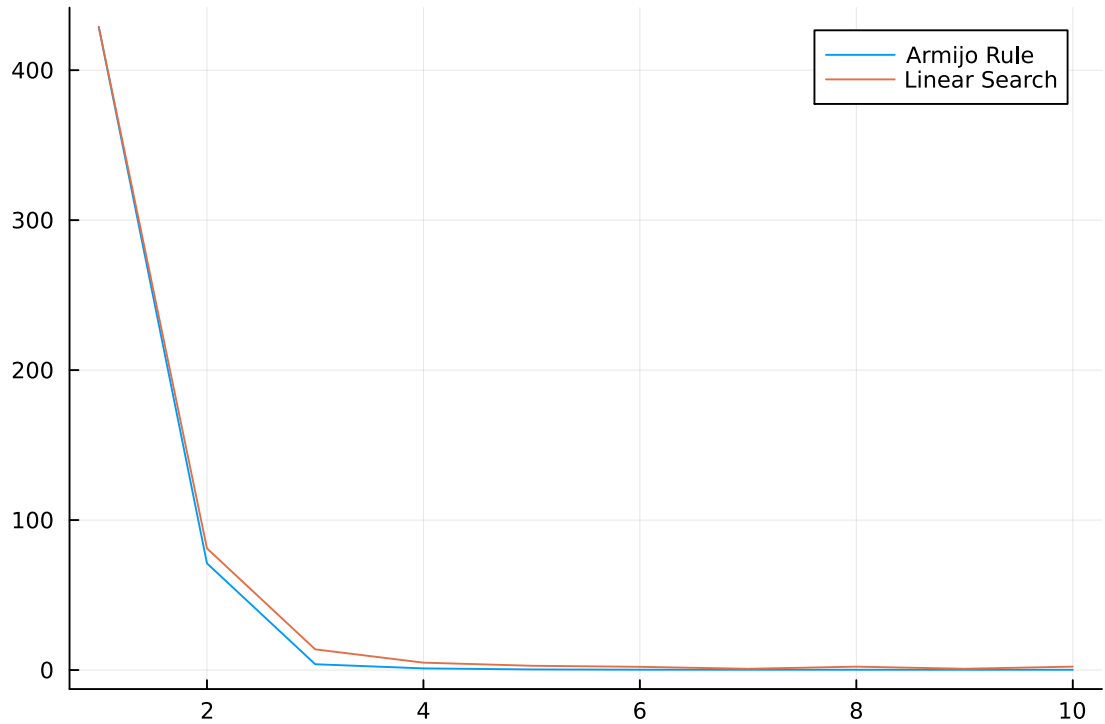
```

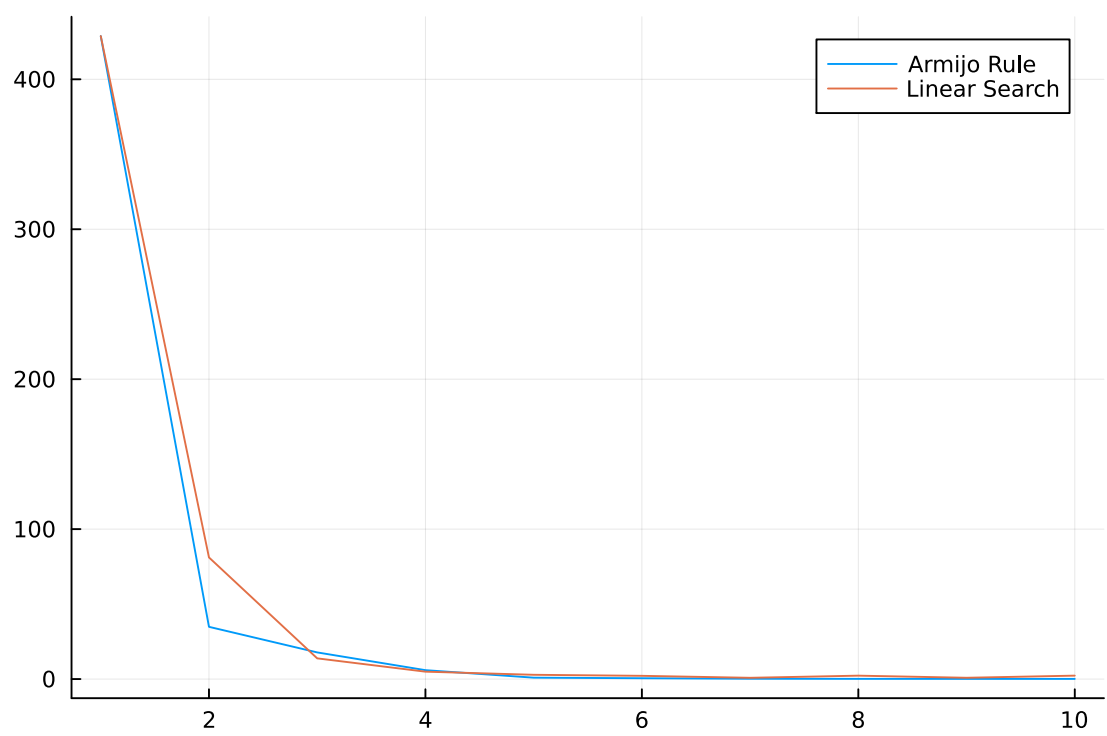
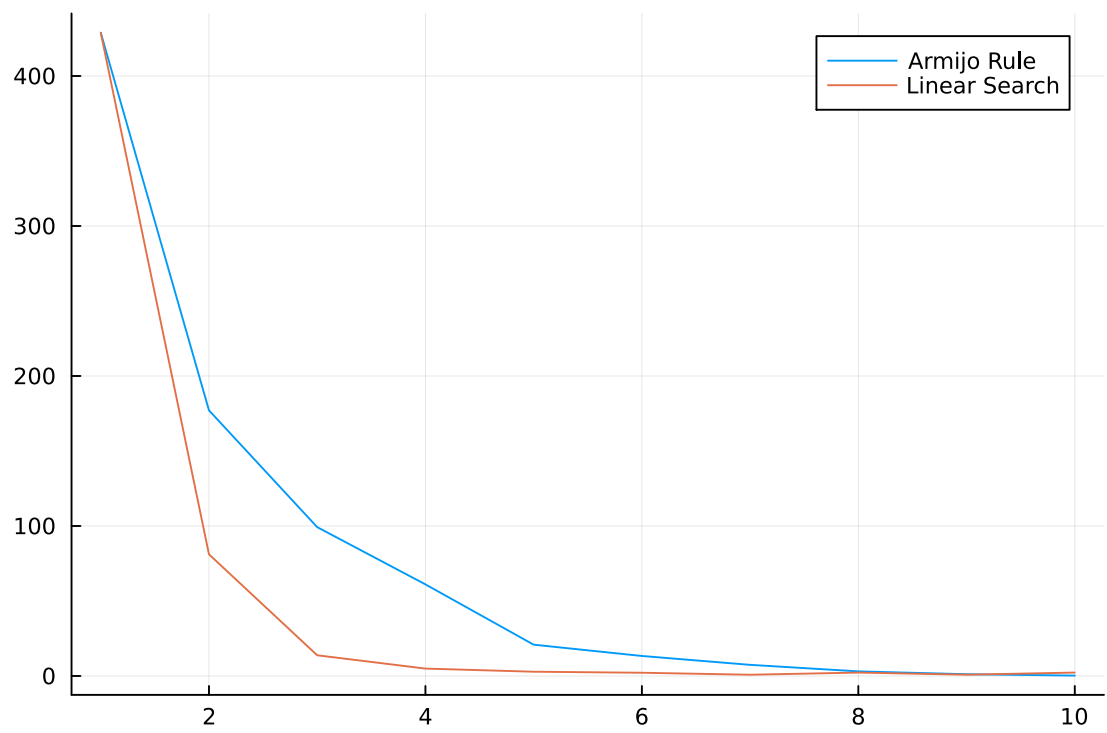
plot_optimization_methods (generic function with 1 method)

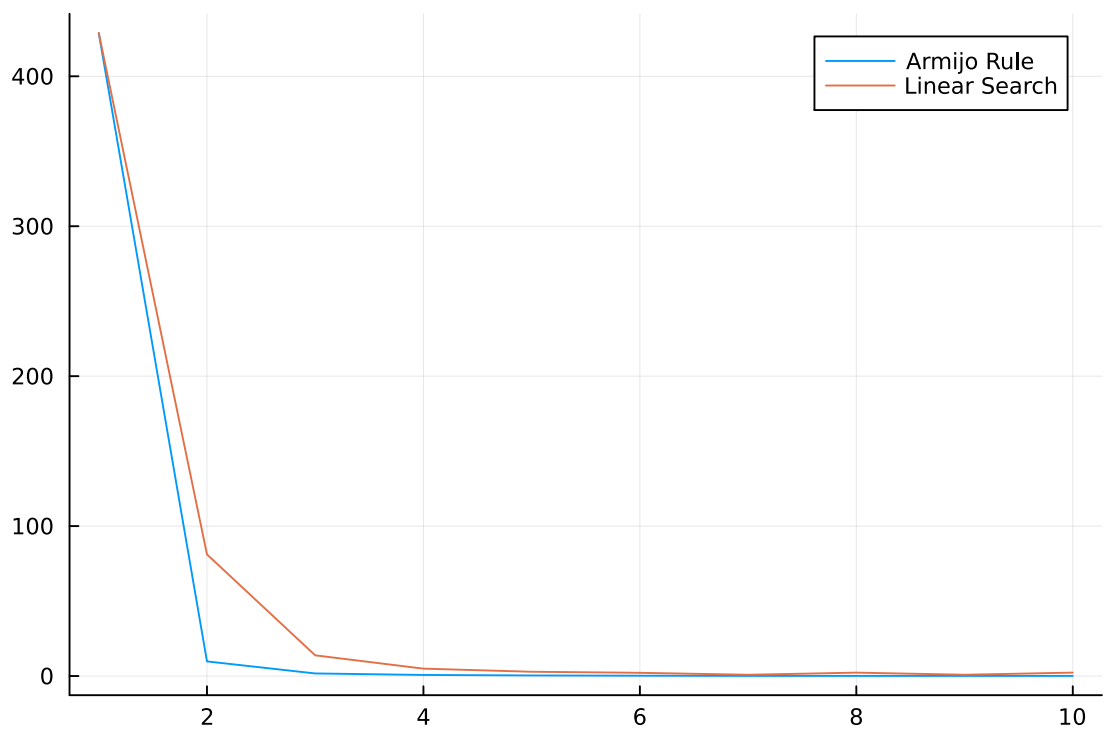
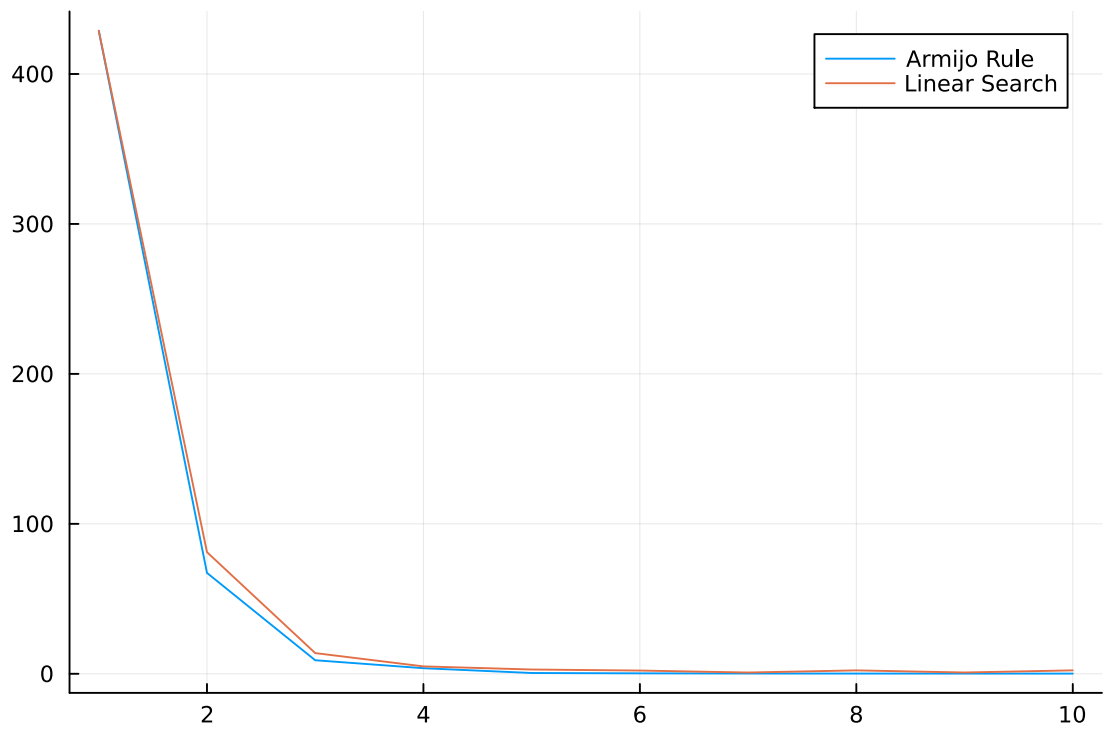
```

In [13]: plot_optimization_methods(0.4, 0.1)
plot_optimization_methods(0.4, 0.5)
plot_optimization_methods(0.2, 0.9)
plot_optimization_methods(0.3, 0.8)
plot_optimization_methods(0.6, 0.6)
plot_optimization_methods(0.9, 0.3)

```







1.9 Bracketing

`bracket_minimum(f, x, s, k, n)` - Funkcja zwracająca przedział, w którym znajduje się minimum funkcji

Argumenty:

- `f` - funkcja
- `x` - wektor współrzędnych początkowych
- `s` - gradient, krok określa jak duży ma być przedział

- k - zwiększa długość przedziału
- n - liczba iteracji

In [14]: **function** bracket_minimum(f, x, s, k, n)

```

a, ya = x, f(x)
b = a .+ s
yb = f(b)

if yb > ya
    a, b = b, a
    ya, yb = yb, ya
    s = .-s
end

for i in 1:n
    c = b .+ s
    yc = f(c)

    if yc > yb
        return a < c ? (a, c) : (c, a)
    end

    a, ya, b, yb = b, yb, c, yc
    s .*= k
end

return 0, 1
end

```

bracket_minimum (generic function with 1 method)

1.10 Przeszukiwanie liniowe z bracketingiem

linear_search_bracketing() - funkcja działa w ten sam sposób jak funkcja linear_search z tą różnicą, że na obliczonym kierunku przez gradient (przekrój funkcji f) dokonujemy automatycznego bracketingu. Wyznaczony bracket dzielimy na mniejsze odcinki, wybierany jest punkt o najmniejszej wartości spośród punktów wyznaczonych przez krótsze odcinki.

Argumenty:

- cost - funkcja f
- grad - gradient funkcji f
- x0 - wektor współrzędnych początkowych
- div - liczba punktów dzielące wyznaczony bracket
- k_ - parametr k, przekazywany do funkcji z bracket_minimum
- n_ - parametr n, bracket_minimum, liczba iteracji

In [15]: **function** linear_search_bracketing(cost, grad, x0, div, k_, n_; max_iter=100, tol
 0 = copy(x0)
 f_values = zeros(max_iter)

```

storage = zeros(length(theta))
for i in 1:max_iter
    value_start = cost(theta)
    storage = grad(theta)
    norm_storage = norm(storage)
    theta_min = typemax(Float64)
    if norm_storage == 0
        break
    end

    bracket_min = bracket_minimum(cost, theta, storage .* (1 / norm_storage), k)
    theta = bracket_min[1]
    theta_new = bracket_min[2]
    for j in 0:div-1
        theta_x = theta + (theta_new - theta) * ((j + 0.5) / div)
        value_x = cost(theta_x)
        if value_x < theta_min
            theta_min = value_x
            theta = theta_x
        end
    end
    value_stop = cost(theta_new)
    f_values[i] = value_start
    if abs(value_stop - value_start) < tol
        break
    else
        theta = theta_new
    end
end
return theta, f_values
end

```

linear_search_bracketing (generic function with 1 method)

1.11 Porównanie metody Armijo z przeszukiwaniem liniowym z bracketingiem

plot_optimization_methods_bracketing(α , p, k, n) - funkcja rysuje wykresy średnich wartości funkcji dla 10 różnych początkowych punktów w kolejnych iteracjach algorytmów. Funkcja przyjmuje argumenty: α i p, które są ustawiane dla optymalizacji z warunkiem wystarczającego spadku Armijo oraz k i n, które są ustawione dla bracketingu.

```

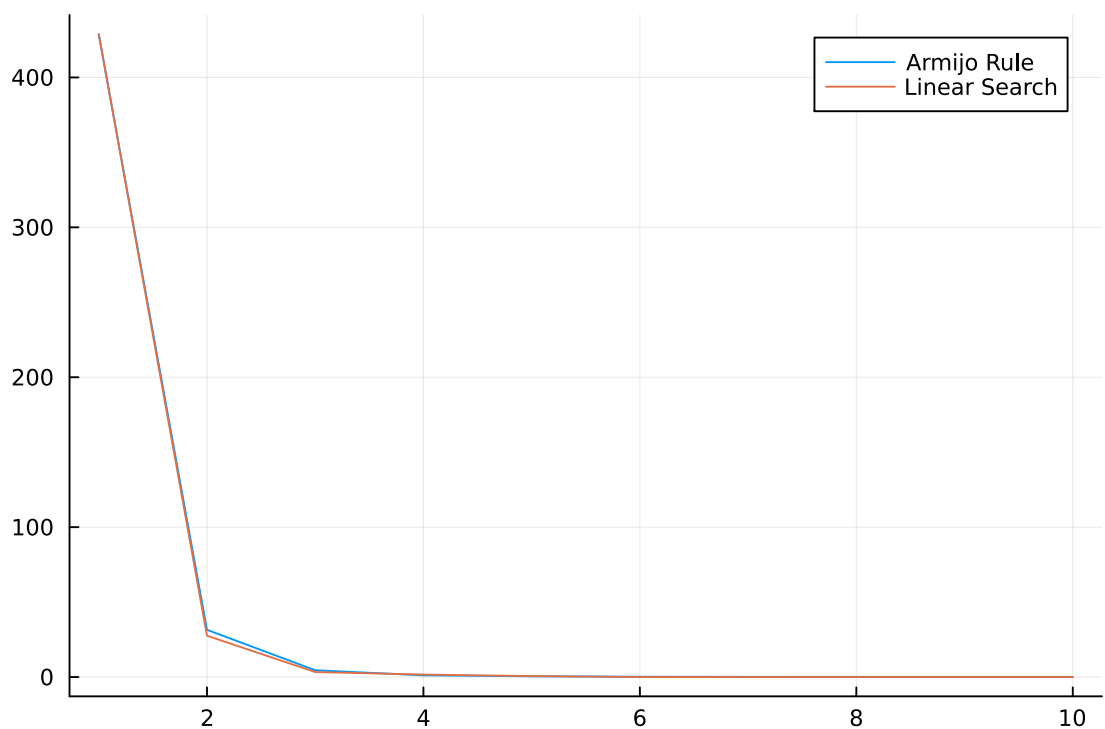
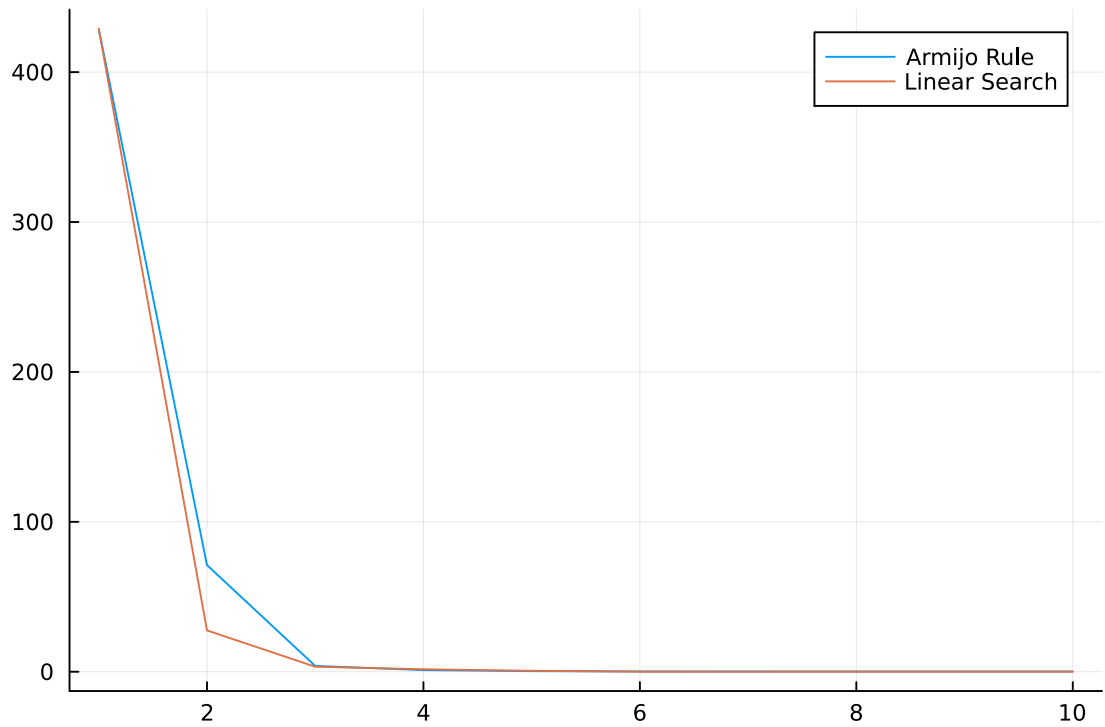
In [16]: function plot_optimization_methods_bracketing(alpha, p, k, n)
    m_armijo = zeros(10)
    m_linear = zeros(10)
    for i in 1:10
        theta_1, f_armijo = steepest_gradient_descent_armijo(f, f_grad, points[i], alpha, p)
        theta_2, f_linear = linear_search_bracketing(f, f_grad, points[i], 10, k, n)
        m_armijo += f_armijo
        m_linear += f_linear
    end
    m_armijo ./= 10
    m_linear ./= 10
    plot(1:length(m_armijo), m_armijo, label="Armijo Rule")
end

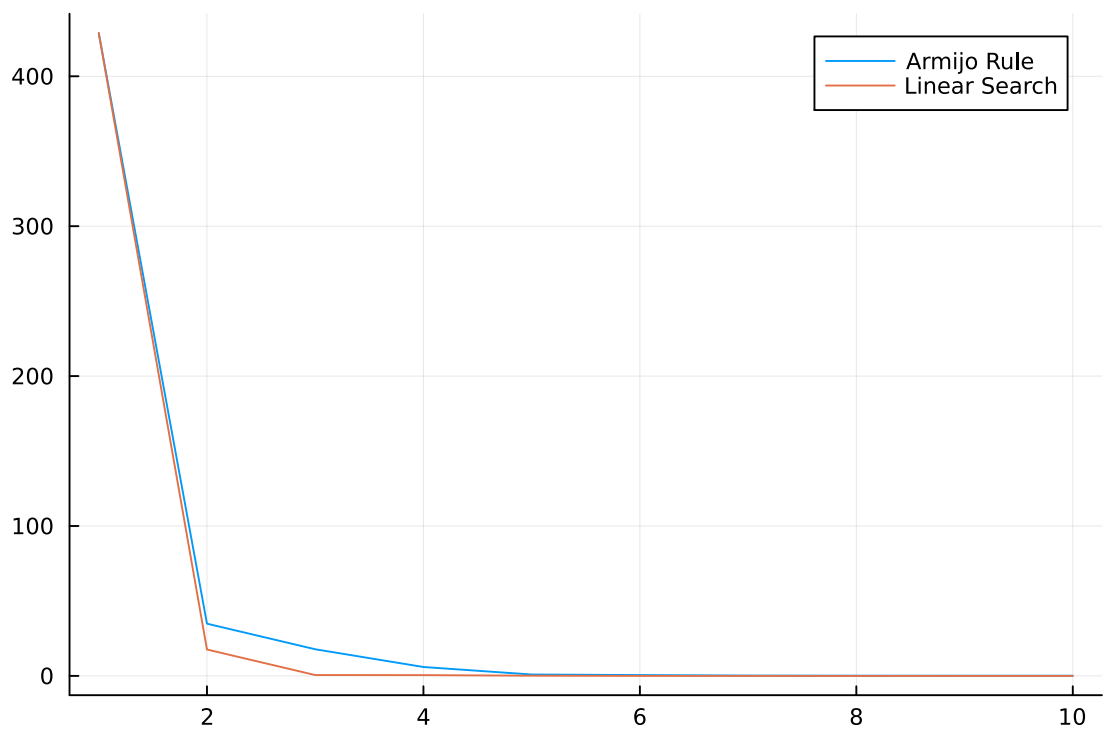
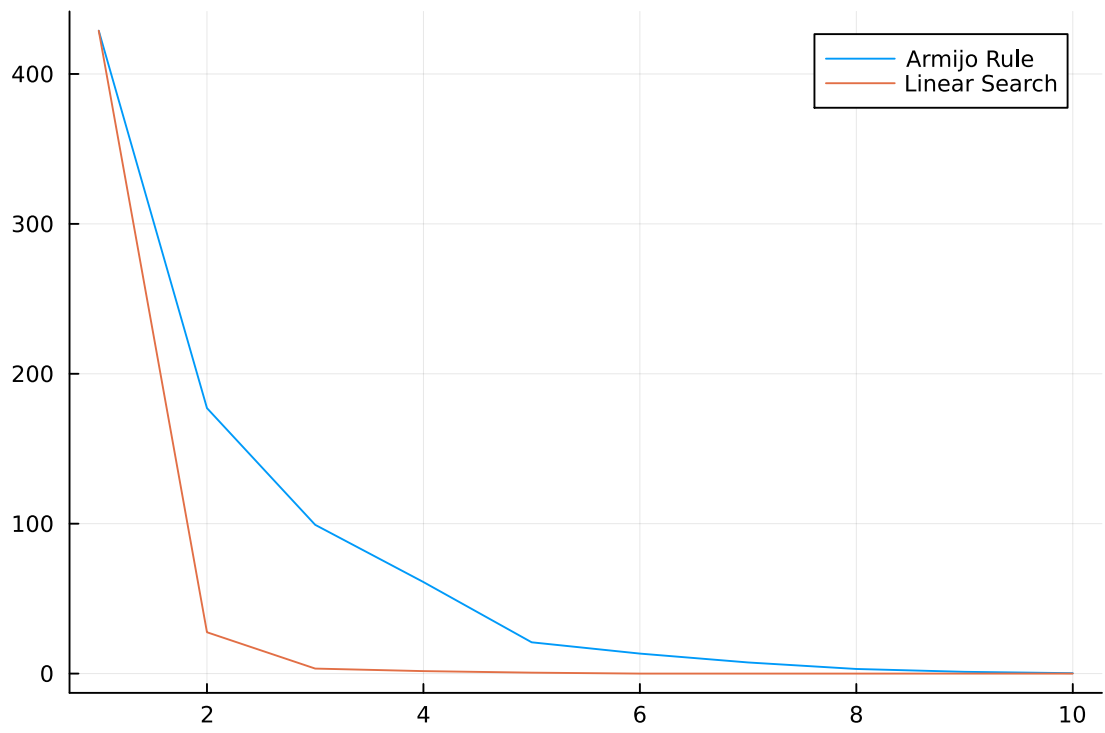
```

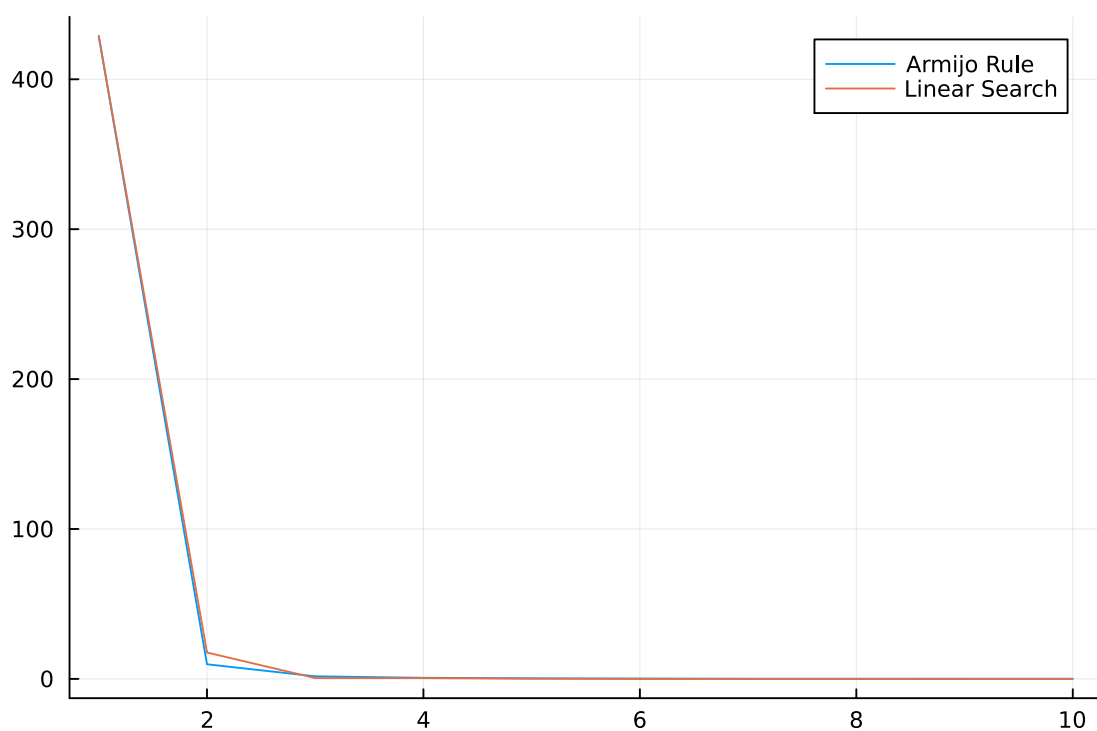
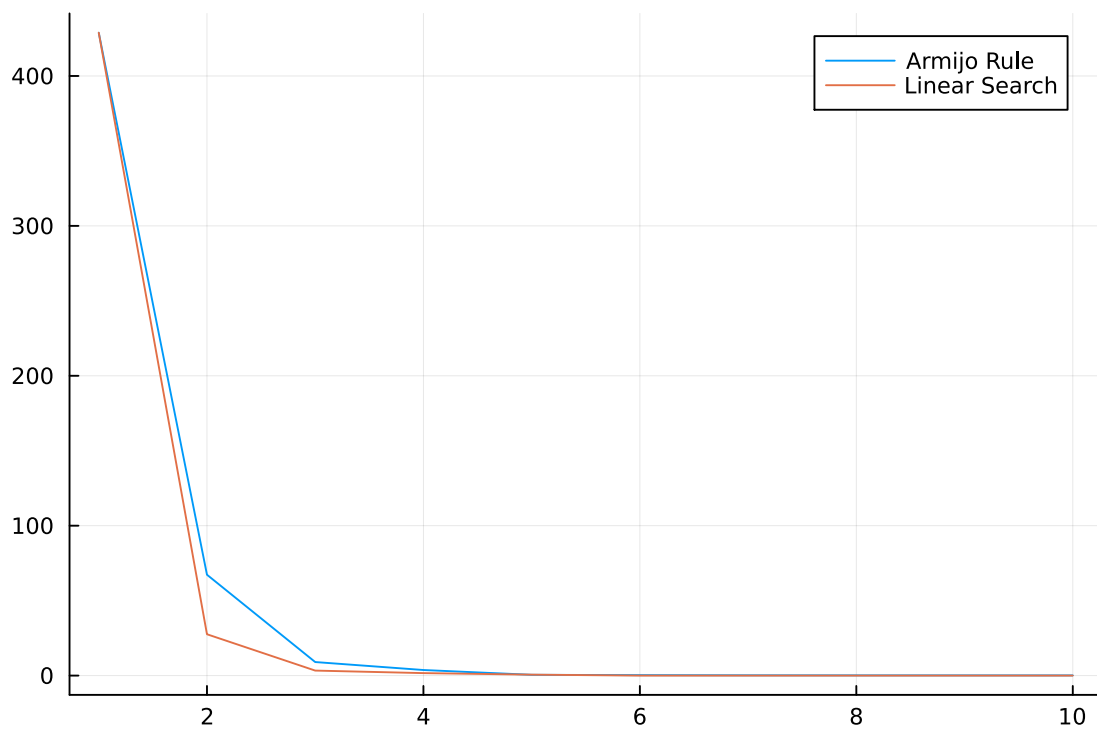
```
display(plot!(1:length(m_linear), m_linear, label="Linear Search"))  
end
```

plot_optimization_methods_bracketing (generic function with 1 method)

```
In [18]: plot_optimization_methods_bracketing(0.4, 0.1, 2, 20)  
plot_optimization_methods_bracketing(0.4, 0.5, 2, 20)  
plot_optimization_methods_bracketing(0.2, 0.9, 2, 20)  
plot_optimization_methods_bracketing(0.3, 0.8, 1, 20)  
plot_optimization_methods_bracketing(0.6, 0.6, 2, 30)  
plot_optimization_methods_bracketing(0.9, 0.3, 1, 20)
```







Wnioski z wykresów

Można zauważyć, że przeszukiwanie liniowe z automatycznym bracketingiem w większości przypadków szybciej prowadzi do znalezienia optimum globalnego.

W niektórych przypadkach Armijo jest lepsze od przeszukiwania liniowego z bracketingiem $[0, 1]$, ponieważ w większości przypadków jest stabilniejsze i szybciej prowadzi do zbieżności.

Natomiast przeszukiwanie liniowe dokładnie znajduje optymalny krok i prowadzi do szybkiej zbieżności globalnej.